



UniSchedule: Machine Learning Based Smart Scheduling for Dynamic Resource Management in University

Supervisor: Dr. P.N.S. Kumara
Co-supervisor: Mrs. K.R.R. Premathilaka

A Thesis Submitted in Partial Fulfilment of the Requirement for the Degree of Bachelor of
Information and Communication Technology
August 2026

W.N.M. Chathuranga (UWU/ICT/21/010)
P.G.U. Dilshan (UWU/ICT/21/013)
S.W.H. Madushan (UWU/ICT/21/032)
S.A. Wellalage (UWU/ICT/21/042)
S.D.N. Silva (UWU/ICT/21/077)

DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

FACULTY OF TECHNOLOGICAL STUDIES

UVA WELLASSA UNIVERSITY

Date of Submission: August 08, 2026

A Thesis for the Degree of Bachelor of Information and Communication
Technology

**UniSchedule: Machine Learning Based Smart
Scheduling for Dynamic Resource Management in
University**

WITHARAMALAGE NADEESH MALAKA CHATHURANGA

PANAKADUWA GAMAGE UDARA DILSHAN

SAMARANAYAKA WIRASINHAGE HARSHA MADUSHAN

SANDALI APSARI WELLALAGE

SENASEEGE DILKI NIMESHIKA SILVA

Department of Information and Communication Technology

Faculty of Technological Studies

Uva Wellassa University of Sri Lanka

August 2026

Declaration

[UWU/ICT/21/010] Chathuranga W. N. M

Signature:

Date 12.08.2026.....

[UWU/ICT/21/013] Dilshan P. G. U

Signature:

Date 12.08.2026.....

[UWU/ICT/21/032] Madushan S. W. H

Signature:

Date 12.08.2026.....

[UWU/ICT/21/042] Wellalage S. A

Signature:

Date 12.08.2026.....

[UWU/ICT/21/077] Silva S. D. N

Signature:

Date 12.08.2026.....

The Above Candidates Have Carried Out Machine Learning Based Smart Scheduling for Dynamic Resource Management in University for the Partial Fulfilment of the Bachelor of Information and Communication Technology Degree Under My Supervision.

Supervisor:

Dr. P.N.S. Kumara

Department of Science and Technology, Uva Wellassa University of Sri Lanka

Signature:

Date.....

Co-supervisor:

Mrs. K.R.R. Premathilaka

Department of Information and Communication Technology, Uva Wellassa University of Sri Lanka

Signature:

Date.....

Approved By:

Approved by:

Final Year Undergraduate Project Coordinator/Department of Information and Communication Technology

Signature:

Date:

Head of the Department/Department of Information and Communication Technology

Signature and official seal:

Date:

Dean/Faculty of Technological Studies

Signature and official seal:

Date:

August 2026

UVA WELLASSA UNIVERSITY OF SRI LANKA

Acknowledgment

We, the undergraduate research team members of group 14, would like to thank all those who helped us to complete our research project successfully “Machine Learning Based Smart Scheduling for Dynamic Resource Management in University” at Uva Wellassa University of Sri Lanka. Their invaluable guidance, technical skills and constant encouragement has been vital in the shaping of this dissertation.

Many thanks to our supervisor, Dr. P. N. S. Kumara, Department of Science and Technology and our co-supervisor, Mrs. K. R. R. Premathilaka, Department of Information and Communication Technology for their valuable suggestions, academic guidance and comments during the course of our research. Their extensive expertise and commitment to excellence in education motivated us to explore the challenges of computational and practical implementation of metaheuristic optimization and predictive machine learning.

We would like to thank Miss. U. W. H. K. Ukgoda, Final Year Undergraduate Research Coordinator of the Department of Information and Communication Technology, for her outstanding facilitation and organizational support, coordination of milestones and resources that has made our research phases go smoothly.

We thank Prof. E. M. U. W. J. B Ekanayake, Acting Head of the Department of Information and Communication Technology for having provided us with the necessary software infrastructure, computational facilities and conducive and collaborative academic atmosphere.

We deeply appreciate the proactive leadership and dedication of Professor K.G.C.Senarathna as the Dean of the Faculty of Technological Studies, in promoting a meaningful research culture in the university.

Last but not least, we are grateful to all our academic staff, administrative staff, colleagues, and families who have supported us in every aspect and been patient with us throughout the entire project with their moral support and encouragement. The creation of this thesis is a result of the teamwork of all.

W. N. M. Chathuranga (UWU/ICT/21/010)

P. G. U. Dilshan (UWU/ICT/21/013)

S. W. H. Madushan (UWU/ICT/21/032)

S. A. Wellalage (UWU/ICT/21/042)

S. D. N. Silva (UWU/ICT/21/077)

Department of Information and Communication Technology

Faculty of Technological Studies

Uva Wellassa University of Sri Lanka

Table of Contents

1.	Introduction.....	1
1.1.	Background and Motivation.....	1
1.2.	Problem Statement.....	1
1.3.	Research Objectives.....	1
1.4.	Research Questions.....	2
1.5.	Proposed Solution in Brief.....	4
1.6.	Scope of the Study.....	4
1.7.	Significance of Study.....	4
1.8.	Structure of the Thesis Document.....	4
2.	Literature Review.....	6
2.1.	Review of Related Work.....	6
2.1.1.	Evolution of Automated University Course Timetabling (UCTP).....	6
2.1.2.	Metaheuristic Optimization Paradigms in Academic Scheduling.....	7
2.1.3.	Multi-Domain Resource Allocation Frameworks (Vehicles & Events).....	8
2.1.4.	Machine Learning and Predictive Demand Forecasting.....	8
2.1.5.	Access Control and Multi-Role University Information Systems.....	9
2.2.	Theoretical Framework.....	9
2.2.1.	Mathematical Formulation of the Scheduling Space.....	9
2.2.2.	Mathematical Taxonomy of Institutional Constraints.....	9
2.3.	Research Gaps.....	10
2.3.1.	Disconnected Multi-Domain Silos in Campus Resource Management.....	11
2.3.2.	Real-Time Dynamic Adaptability Deficits.....	11
2.3.3.	Computational Scaling Limitations in Highly-Coupled Curriculum Networks.....	11
2.4.	Comparative Analysis of Existing Approaches vs. UniSchedule.....	12
2.5.	Summary of Key Findings from Literature.....	13
3.	Materials and Methods.....	14
3.1.	Overview and System Architecture.....	14
3.2.	Mathematical Formulation of Timetabling Constraints.....	16
3.3.	GA Engine Implementation.....	17
3.4.	Automated Conflict Resolution and AI Suggestion Mechanism.....	19
3.5.	Machine Learning Vehicle Demand Forecasting.....	23
3.5.1.	Feature Engineering and Pipeline Sequence.....	24
3.6.	Performance and Evaluation Metrics.....	24
4.	Technology Consideration.....	25
4.1.	Frontend Client Layer Architecture.....	25
4.1.1.	React.js and Virtual DOM Optimization.....	25
4.1.2.	Responsive Utility-First Presentation with Tailwind CSS.....	26
4.2.	Backend RESTful API Layer Architecture.....	26
4.2.1.	Asynchronous Event-Driven Concurrency.....	26
4.2.2.	Strict Data Validation with Pydantic.....	27
4.3.	Persistence and Object-Relational Mapping (ORM) Layer.....	27
4.3.1.	Relational Integrity and Junction Engineering.....	27
4.3.2.	SQLAlchemy Mapping and Connection Pooling.....	27
4.4.	Computational Optimization Core.....	28
4.4.1.	Object-Oriented Genetic Programming.....	28
4.4.2.	Evolutionary Operator Customization.....	28

4.5.	Predictive Analytics and Machine Learning Engine	29
4.5.1.	High-Performance Data Manipulation with Pandas	29
4.5.2.	Robust Predictive Modelling with Scikit-learn	29
4.6.	Cloud Infrastructure and Deployment	29
4.6.1.	Cloud Infrastructure and Services Breakdown	29
4.6.2.	Key Advantages of the Deployment Architecture	30
5.	Development, Testing, Implementation and Evaluation	31
5.1.	System Development Workflows and Feature Implementation	31
5.1.1.	Full-Stack System Architecture and Modular Component Breakdown	31
5.1.2.	Relational Database Normalization and Junction Schema Engineering	31
5.1.3.	Role-Based Access Control (RBAC) and User Portals	32
5.2.	Comprehensive Software Testing Framework and Results	32
5.2.1.	Overview of System Test Suite Execution	32
5.2.2.	Portal-Wise Test Execution Metrics and Analysis	33
5.2.3.	Representative Test Cases Across Functional Modules	33
5.2.4.	Defect Tracking, Root-Cause Analysis, and Resolution Summary	35
5.3.	System Implementation and Deployment Architecture	35
5.3.1.	Containerized Infrastructure Configuration	36
5.4.	Quantitative Performance and Experimental Evaluation	36
5.4.1.	Genetic Algorithm Convergence and Constraint Optimization	36
5.4.2.	Machine Learning Vehicle Demand Forecasting Performance	37
6.	Results and Discussion	38
6.1	Implementation and Evaluation of the ML-Based Vehicle Demand Forecasting Model	38
6.1.1	Overview of the Predictive Model	38
6.1.2	Data Preprocessing and Feature Engineering	38
6.1.3	Model Training and Performance Evaluation	39
6.1.4	Integration and Dashboard Presentation Layer	39
6.1.5.	Practical Implications and Operational Impact	40
6.2.	Implementation and Evaluation of the Genetic Algorithm (GA) Timetable Optimization Engine	40
6.2.1.	Algorithm Design and Constraint Formulation	40
6.3.	System-Wide Comparative Analysis and Utilization Impact	40
6.3.1.	Resource Utilization Rate (RUR) Evaluation	40
6.3.2.	Practical Operational Impact	41
7.	Conclusion and Recommendations	42
7.1.	Conclusion	42
7.2.	Recommendations	43
7.2.1.	Institutional Deployment and Change Management	43
7.2.2.	System Maintenance and Model Governance	43
8.	Future Perspectives	44
8.1.	Incorporation of Academic Calendar Features into ML Pipelines	44
8.2.	Hybridization of Metaheuristic Solvers	44
8.3.	Automated Push Notifications and Mobile Native Integration	45
9.	References	46
10.	Appendices	47
	Appendix A: Core Algorithmic & Backend Implementation Code	47

Figure A.1: Genetic Algorithm Fitness Function Initialization and Constraint Weighting (evaluate function)	47
Figure A.2: Evaluation Logic for Consecutive Session Penalties and Location Checks	47
Figure A.3: Configuration of the DEAP Evolutionary Toolbox and GA Execution Pipeline (run_ga).....	48
Figure A.4: Vehicle Demand Prediction Data Preprocessing and Feature Engineering ...	48
Figure A.5: Multi-Output Random Forest Model Training, Test Split, and Serialization .	49
Figure A.6: SQLAlchemy ORM Model Mapping for Timetable Sessions (timetable_sessions).....	49
Appendix B: Role-Based User Portal Dashboards	50
Figure B.1: System Administrator Analytics Dashboard	50
Figure B.2: Lecturer Portal Dashboard.....	50
Figure B.3: Resource Manager Facility and Fleet Dashboard.....	50
Figure B.4: Student Portal Academic Dashboard.....	51
Appendix C: AI Timetabling & Real-Time Conflict Resolution Interfaces	51
Figure C.1: AI Timetable Generator Parameter & Grid Interface	51
Figure C.2: Real-Time AI Conflict Interception & Suggestion Modal.....	51
Appendix D: Machine Learning Demand Forecasting Analytics	52
Figure D.1: Vehicle Demand Forecast Dashboard and Distribution Metrics	52
Appendix E: Software Quality Assurance and Test Case Audit Logs.....	52
E.1: Comprehensive System Test Case Audit Suite.....	52

List of Tables

Table 1: Summary of Identified Literature Research Gaps	11
Table 2: Comparative Matrix of Resource Scheduling Approaches and Technologies	12
Table 3: Experimental and Development Environment Specifications	15
Table 4: High-Level Full-Stack System Topology and Tier Interactions	31
Table 5: Empirical Portal-Wise Software Testing Audit Summary	33
Table 6: Containerized Microservices Deployment Topology.....	35
Table 8: Machine Learning Model Evaluation Metrics.....	39

Table of Figures

Figure 1: High-Level System Architecture.....	14
Figure 2: Use Case Diagram of the UniSchedule System.....	16
Figure 3: Genetic Algorithm (GA) Process Flow for Automated Timetable Generation.....	18
Figure 4: AI Conflict Interceptor Process for Conflict Detection and Resolution.....	20
Figure 5 : Machine Learning Demand Forecasting Pipeline for Vehicle Demand Prediction ..	21
Figure 6: Activity Diagram for Resource Request Processing and AI Conflict Interception	22
Figure 7: Activity Diagram for ML-Based Vehicle Demand Forecasting Workflow	23
Figure 8: Technological Layers	25
Figure 9: Test Cases for the Admin Portal.....	33
Figure 10 : Test Cases for the Lecturer Portal	34
Figure 11: Test Cases for the Resource Manager Portal.....	34
Figure 12: Test Cases for the Scheduler Portal (Initial failed test case was resolved and successfully retested).....	34
Figure 13: Test Cases for the Student Portal (Initial failed test case was resolved and successfully retested).....	34
Figure 14: Genetic Algorithm Convergence Trend over 160 Generation.....	36

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
AWS	Amazon Web Services
CDN	Content Delivery Network
CLP	Constraint Logic Programming
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CRUD	Create, Read, Update, Delete
CSOP	Constraint Satisfaction and Optimization Problem
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DEAP	Distributed Evolutionary Algorithms in Python
DOM	Document Object Model
DRL	Deep Reinforcement Learning
DSR	Design Science Research
EC2	Elastic Compute Cloud
GA	Genetic Algorithm
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
ILP	Integer Linear Programming
JSON	JavaScript Object Notation
JWT	JSON Web Token
MAE	Mean Absolute Error
ML	Machine Learning
OAuth	Open Authorization
ORM	Object-Relational Mapping
PSO	Particle Swarm Optimization
QA	Quality Assurance
RBAC	Role-Based Access Control
RDS	Relational Database Service
REST	Representational State Transfer

RL	Reinforcement Learning
RUR	Resource Utilization Rate
SA	Simulated Annealing
SPA	Single Page Application
SQL	Structured Query Language
SSL	Secure Sockets Layer
TS	Tabu Search
UCTP	University Course Timetabling Problem
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience
3NF	Third Normal Form

Abstract

Machine Learning Based Smart Scheduling for Dynamic Resource Management in University

Chathuranga W. N. M.

Dilshan P. G. U.

Madushan S. W. H.

Wellalage S. A.

Silva S. D. N.

Department of Information and Communication Technology

Faculty of Technological Studies

Uva Wellassa University of Sri Lanka

Supervisor: Dr. Kumara P. N. S.

Co-supervisor: Mrs. Premathilaka K. R. R.

Operating resource networks with multiple interdependencies, such as academic class timetables, venue reservations, and vehicle fleets, in isolation using manual compilation spreadsheets presents severe operational challenges for modern higher education institutions. These legacy processes lead to frequent double-bookings, asset underutilization, high administrative costs, and rigid institutional structures. To address these challenges, this study presents a unified full-stack web ecosystem built upon a decoupled architecture, known as **"UniSchedule,"** featuring secure Role-Based Access Control (RBAC) portals for administrators, lecturers, schedulers, resource managers, and students, supported by an optimized relational database infrastructure.

The main processing core runs a background Genetic Algorithm (GA) built with the Distributed Evolutionary Algorithms in Python (DEAP) software library. This engine automatically generates working weekly timetables by strictly following absolute university rules while keeping schedules as comfortable as possible for both staff and students. For daily flexibility, the system wraps this scheduling engine in a live Artificial Intelligence (AI) conflict guard. This tool tracks manual changes as they happen, instantly checks them against database rules, and immediately suggests alternative, conflict-free rooms or times right when the user tries to make a change. Most importantly, the system includes an intelligent predictive module driven by Machine Learning (ML) regression pipelines to handle predictive demand forecasting. By analyzing past university transit data, this module forecasts future transportation demand peaks and vehicle shortage bottlenecks ahead of time, allowing resource managers to optimize fleet allocation and manage vehicle bookings proactively before scheduling conflicts can happen.

According to system evaluations, the platform significantly cuts down on scheduling mistakes, removes manual coordination delays, and helps the university use its classrooms and vehicles much better. Overall, this project offers a scalable, reusable blueprint for university digital upgrades, successfully organizing workflows across academic schedules, transit services, and events while reducing the need for separate software tools.

Keywords: university course timetabling, genetic algorithm, machine learning, conflict resolution, resource optimization, full-stack framework, predictive demand forecasting

1. Introduction

1.1. Background and Motivation

The digital age has revolutionized the way organizations operate on a day-to-day basis. Historically, university scheduling was handled manually, by creating a fixed schedule and spreadsheet. Higher education institutions are expanding rapidly and becoming increasingly complex. To keep up with hundreds of lessons, various groups of students, specialized classrooms and the overworked lecturer is a constant challenge for modern universities. Manually, this process is time consuming, susceptible to scheduling errors and cannot adapt to unexpected changes quickly. This project is driven by these operational pain points in the management of university infrastructure, with the goal of providing a complete digital infrastructure, using computational intelligence to automate, optimize and streamline the allocation of institutional resources.

1.2. Problem Statement

The sheer volume of disconnected resources across universities presents a significant challenge for effective management. Managing a multitude of disconnected resources such as faculty schedules, classroom availability, university event bookings, and a university's vehicle fleet presents a significant challenge for universities. The ability to treat these domains separately, by department, is a traditional practice that often means that a change in one area leads to imponderable cascading conflicts in another. For instance, moving a classroom could conflict with a forthcoming event reservation, or a lecturer's availability. Similarly, uncoordinated vehicle bookings create severe transit bottlenecks for academic field trips and administrative events. Manually, resolving these issues adds a heavy administrative burden, underutilizes resources, and impacts the student and staff experience. Thus, the need for a unified platform supported by an intelligent optimization engine and predictive analytics is essential to create a conflict-free environment.

1.3. Research Objectives

The main aim of this research is to develop an automated, intelligent and conflict-free resource management ecosystem known as “UniSchedule”. To meet this goal, the project has the following objectives:

- To design and develop a web-based scheduling application featuring distinct, secure portals for Administrators, Lecturers, Schedulers, Resource Managers and Students using a RBAC model.
- To formulate and implement a robust GA framework utilizing the DEAP library to generate conflict-free university timetables adhering to rigid physical and operational constraints.

- To engineer an interactive, real-time AI Conflict Resolution Module capable of intercepting conflicting booking attempts and instantly suggesting alternative physical venues and time-slots.
- To develop and integrate a ML predictive model utilizing Random Forest pipelines to execute predictive demand forecasting for the university vehicle fleet based on historical institutional transit data.
- To integrate crucial student service automation sub-modules including secure semester registration, and an administrative approval workflow for medical submissions.

1.4. Research Questions

The following are the major research questions in this study that will support the development and evaluation of the system:

1. What method can be used to synchronize a complex and dynamic multi-domain schedule requirement (such as an academic timetable, event spaces, or a vehicle fleet) within a single relational database and optimization model?

- The various domains' scheduling needs are synchronized using a normalized MySQL relational database schema (Third Normal Form (3NF)) with specialized junction tables (e.g. module_degrees) for the management of complex and shared resource dependencies.
- This schema is connected to FastAPI via SQLAlchemy Object-Relational Mapping (ORM), and supports connection pooling for parallel academic, event, and transit reads and writes.
- The core optimization engine is able to abstract schedules into multi-dimensional matrix structures, which allows it to evaluate the availability of resources across departments at the same time, avoiding the silos of isolated departments.

2. How to implement the constraint prioritization technique using the genetic algorithm technique? How well can a metaheuristic method, such as a Genetic Algorithm (GA), efficiently select between absolute hard constraints and variable soft constraints, in a dense educational ecosystem?

- In the Distributed Evolutionary Algorithms in Python (DEAP) evolutionary framework, the GA takes absolute hard constraints as its top priority, giving heavy mathematical penalties to chromosomes that break non-negotiable constraints (e.g., lecturer double booking, room overlap, cohort collisions).
- Soft constraints (e.g., minimize gap times, respect preferred hours) are penalized with lower penalty weights.

- The results of the experiments demonstrate that in 300 generations the engine reaches 100.00% Hard Constraint Satisfaction Rate, while keeping an average of 88.50% Soft Constraint Satisfaction Rate.

3. How to use historical scheduling and resources allocation to effectively implement predictive demand forecasting for future institutional vehicle allocations based on ML models?

- The pre-processing of the historical trip log, are performed using the Pandas library to extract important temporal data (day of week, month, weekend indicator).
- The Multi-Output Random Forest Regressor model is created using approved historical requests and simultaneously predicts daily demand for five classes of vehicles – Cars, Vans, Mini Buses, Buses, Trucks.
- With a Mean Absolute Error (MAE) of 0.40 Vehicles, the model can help resource managers predict fleet shortages beforehand, enabling them to make proactive adjustments to allocations before booking conflicts.

4. What are the most effective UI/UX architectural approaches to visualize complex, multi-layered timetables and capture real-time availability data from institutional stakeholders without cognitive overload?

For complex, multi-layered timetables, the best strategy for providing a great UI/UX experience is to create a Single Page Application (SPA) that uses React.js and Tailwind CSS.

- **Virtual DOM Reconciliation:** React's Virtual Document Object Model (DOM) pipeline prevents browser rendering lag and visual stutter during interactive drag-and-drop operations, by isolating and re-rendering only the cells that have been changed.
- **Responsive Grid Adaptability:** Tailwind CSS offers a flexible, mobile-first responsive design, collapsing the wide grid containing multiple columns into clean, single column cards on smaller mobile screens for a seamless user experience across desktop and mobile devices without clutter.

5. How to enable real-time dynamic interception of manual override? What are the possibilities for dynamically intercepting and validating real time manual administrative overrides without having to re-execute full of the population evolution algorithm?

- Dynamic manual overrides by an AI Conflict Interceptor module in the application layer.
- If an ad-hoc schedule change is requested by the user, then the interceptor will run a real-time check on a decision-tree built against current database rules. If a constraint violation is identified, the operation is blocked and the module asks for available time or space slots and

returns immediate recommendations to the UI without having to rerun the entire GA generation process, generating recommendations that are not in conflict.

1.5. Proposed Solution in Brief

The solution, known as "UniSchedule," is a full stack smart production-ready scheduling application. It integrates all admin and user actions into a single environment, with its front-end built on React.js, CSS powered by Tailwind, and its back-end using FastAPI (Python) and a MySQL database. The core of the system is a GA engine based on the package DEAP that automates the generation of timetables using multi-variable fitness functions. It also includes an instant AI Conflict Guard that validates manual overrides on the fly and provides intelligent alternate recommendations. Crucially, the system features a ML predictive layer using Random Forest regression pipelines to provide predictive demand forecasting. By analysing past institutional transit datasets, this module forecasts future transportation demand peaks and vehicle shortage bottlenecks, allowing administrators to manage vehicle booking proactively before conflicts occur.

1.6. Scope of the Study

The scope is particular to the faculty-level operational workflows of Uva Wellassa University of Sri Lanka (UWU). The system simulates the physical infrastructure such as regular lecture halls, special computer laboratories, lecturer availability limitations, student registration procedures, and approval tracking for auxiliary events and vehicles. The academic schedule at the faculty level is the key element in the core optimization, where structured expansion interfaces are provided for transport and asset management units using ML forecasting.

1.7. Significance of Study

This study has practical and academic impact on the national university systems. Functionally, it helps reduce the huge administrative burden of effective semesters, boosts the RUR of the university, and provides predictive insights for vehicle fleets while avoiding conflicting schedules for vehicle fleets while avoiding conflicting schedules for students and staff. Academically, it has created a framework for the deployment of the metaheuristic scheduling models inside complex public institutions within Sri Lanka and set a baseline for localizing the software deployment and digital transformation within public institutions.

1.8. Structure of the Thesis Document

This thesis is systematically organized into five comprehensive chapters, tracing the lifecycle of the project from inception to implementation and evaluation:

- **Chapter 1: Introduction** establishes the administrative baseline, localized problem statement, research goals, questions, scope, and institutional significance.
- **Chapter 2: Literature Review** carries out an extensive review of traditional, operational research, and ML methodologies, mapping out hard and soft constraint definitions.
- **Chapter 3: Materials and Methods** discuss the underlying methodology (Design Science Research), data collection steps, and relational design blocks, including the complete database data dictionary.
- **Chapter 4: Technology Consideration** explores the specific software stack (React.js, FastAPI, MySQL, DEAP library) and structural performance configurations.
- **Chapter 5: Development, Testing, Implementation and Evaluation** presents the implemented system architecture, user portals, completed features, development challenges, system testing, implementation process, and evaluation results.

2. Literature Review

The management of physical, human, and auxiliary resources represents a complex operational challenge within higher education institutions. As modern universities expand their academic curricula, degree offerings, and student enrolment volumes, the allocation of limited resources, such as specialized lecture halls, computer laboratories, spaces for university events, and official vehicle fleets, rapidly evolves into combinational optimization problem. Traditionally, institutional administration relies on manual compilation techniques, and static spreadsheets for resource planning and scheduling. However, these legacy approaches provide limited capability for dynamically adapting to live operational disruptions, resulting in resource underutilization, increased administration costs, and constant scheduling conflicts.

This chapter critically reviews the development of automated timetabling, metaheuristic optimization paradigms, multi-domain resource allocation, role-based security architectures and predictive machine learning algorithms to provide the theoretical foundation for the proposed UniSchedule system. This review synthesizes the available literature to identify key research gaps in current institutional scheduling solutions and justifies the design aspects, algorithmic selections, and system decoupling approach of the UniSchedule ecosystem.

2.1. Review of Related Work

2.1.1. Evolution of Automated University Course Timetabling (UCTP)

The University Course Timetabling Problem (UCTP) is a well-known NP-hard combinatorial optimization problem in which a set of academic events (lectures, laboratory sessions, courses) must be assigned to a defined spatiotemporal grid, consisting of physical rooms and operational time-slots. Previous studies by Schaerf[1] and Lewis[2] have shown that classical manual scheduling approaches become increasingly ineffective as student enrollment and structural constraints grow. Furthermore, Babaei et al.[3]demonstrated that manual timetabling is time-consuming, error-prone, and unstable to scale effectively in institutions with multiple student cohorts and shared faculty members.

To automate this process, operational research initially focused on deterministic mathematical programming techniques, such as Integer Linear Programming (ILP) and Constraint Logic Programming (CLP). Daskalaki et al.[4], investigated exact 0-1 integer programming formulations for university timetabling problems. However, the computational complexity of these approaches increase significantly as the number of decision variables and constraints grows, thereby limiting their scalability in large real world scheduling environments. In complex institutional settings involving hundreds of course modules, multi-departmental degree programs, and varying room capacities, deterministic solvers experience combinatorial explosion, leading to excessive computation times and reduced practicality for large scale institutional implementation. Consequently, research shifted toward stochastic, metaheuristic

optimization techniques capable of producing near-optimal solutions within acceptable computational timeframes.

2.1.2. Metaheuristic Optimization Paradigms in Academic Scheduling

Metaheuristics provide robust frameworks for navigating vast, non-linear search spaces by balancing global exploration (searching unvisited regions of the solution space) local exploitation (refining promising solutions). A broad range of metaheuristic paradigms has been applied to the UCTP including Genetic Algorithms (GA), Simulated Annealing (SA), Tabu Search (TS), and Particle Swarm Optimization (PSO).

Genetic Algorithm (GA)

Genetic Algorithm frameworks, grounded in the principles of biological evolution and natural selection, represent potential timetable configurations as mathematical arrays known as chromosomes. Through iterative generations involving selection, crossover (recombination), and mutation operators, GA-based approaches evolve a population of candidate schedules toward an optimal fitness score. Burke et al. [5], demonstrated the effectiveness of memetic evolutionary approaches in examination timetabling problems, showing that population-based search methods are well suited for dense constraint environments. Sutar and Bichkar [6] showed that GA performs effectively in global search by exploring complex constraint landscapes without becoming trapped in local optima. Pillay [7] further validated the flexibility of GA chromosome representations in accommodating diverse academic rules across educational institutions. However, standard GA variants may exhibit slow convergence in the later stages of the search process if crossover operators disrupt highly fit sub-structures, commonly referred to as building blocks, within the schedule.

Tabu Search (TS) and Local Search Neighborhoods

TS is a memory-based local search technique that temporarily prevents recently visited solutions from being revisited by maintaining a tabu list. The mechanism enables the algorithm to escape local optima and explore new regions of the search space. Hertz and de Werra [8] successfully applied neighborhood search and tabu mechanisms to solve graph coloring, and timetabling-related scheduling problems. Although TS provides strong local exploitation capabilities, its effectiveness largely depends on the quality of the initial seed schedule. Poor initial solutions may lead to slower convergence and lower quality schedules, particularly in large-scale timetabling problems.

Hybrid Metaheuristic Approaches

In recent years, research has shifted to hybrid metaheuristic approaches to overcome the limitations of individual optimization techniques. Combining global search algorithms, such as GA with the local search heuristics, such as TS or Hill climbing, has consistently improved convergence stability and Constraint

Satisfaction Rates (CSR) across benchmark UCTP datasets. Wong et al.[9], demonstrated that GA-based approaches are effective for solving real-world university course timetabling problems.

2.1.3. Multi-Domain Resource Allocation Frameworks (Vehicles & Events)

An important weakness in the current body of academic scheduling research is the separation of resource domains. The legacy optimization tools are designed primarily for classroom and lecture hall assignments, treating auxiliary university operations, such as institutional vehicle fleet allocation and event space reservations, as separate administrative silos.

Real-world educational institutions involve multiple stakeholders competing for shared physical resources and institutional budgets to support schedules, extracurricular activities, and official transportation requirements. Managing these domains through uncoordinated, independent software systems results in Silent cross-departmental scheduling conflicts. For example, an academic lecture hall may be scheduled simultaneously for a university symposium, or an off-campus field trip may be planned without the availability of transportation vehicles. Zhou et al.[10], proposed composite reward mechanisms for manufacturing environments; however, the underlying concept of integrated resource optimization is transferable to resolving institutional asset competition in university management systems.

2.1.4. Machine Learning and Predictive Demand Forecasting

Metaheuristic solvers are highly effective for static, batch-generated schedule optimization; however, they are inherently non-predictive and reactive. They optimize schedules based strictly on fixed input parameters provided at the beginning of a semester. However, university auxiliary operations, particularly transit requests and vehicle fleet allocations, are subject to dynamic, non-linear seasonal fluctuations driven by academic calendars, field excursions, and examination periods.

Recent advancements in predictive analytics have led to the integration of Regression Pipelines built with Machine Learning (ML) into resource management systems. Breiman [11] demonstrated that Random Forest ensemble models are effective for high-dimensional prediction tasks because they can capture non-linear relationships without making strict parametric assumptions. By analysing historical usage logs, cohort sizes and temporal features such as day of week, academic week and month, Random Forest regression models can support the prediction of future demand patterns and potential fleet shortages weeks in advance, enabling administrators to transition from reactive booking approaches to proactive asset allocation strategies.

2.1.5. Access Control and Multi-Role University Information Systems

Modern educational software relies on centralized databases to interact with a variety of end users, such as administrators, schedulers, academic staff, resource managers, and students. Ferraiolo et al. [12], introduced Role-Based Access Control (RBAC) as the de facto standard for enforcing security, the principle of least privilege and role separation in large enterprise systems. With the adoption of RBAC in modern university management systems, the academic staff as well as students are provided with dedicated and simplified views, while the administrative functions such as triggering timetable generation and modifying master room data are restricted to authorized schedulers. Decoupled RBAC architectures ensure data integrity and establish clear operational boundaries across multi-portal campus applications.

2.2. Theoretical Framework

2.2.1. Mathematical Formulation of the Scheduling Space

The UCTP is formally classified as a multi-dimensional constraint satisfaction and optimization problem. The institutional search space ($\mathbf{S}_{\text{space}}$) is modeled as a discrete Cartesian product of four primary operational sets:

$$\mathbf{S}_{\text{space}} = \mathbf{T} \times \mathbf{S} \times \mathbf{L} \times \mathbf{R} \quad [2.1.]$$

Where:

- $\mathbf{T}$ represents the set of discrete weekly operational time-slots.
- $\mathbf{S}$ represents the set of distinct cohorts of student batches and degree pathways.
- $\mathbf{L}$ represents the set of available academic lecturing staff.
- $\mathbf{R}$ represents the set of physical infrastructure assets, including lecture halls and computer laboratories and event venues.

A single candidate timetable individual (chromosome) $\mathbf{X}$ is represented as a structured chromosome mapping academic events to combinations of operational resources. The objective of the evolutionary engine is to navigate $\mathbf{S}_{\text{space}}$ to maximize the overall Resource Utilization Rate (RUR) while reducing constraint penalties to zero.

2.2.2. Mathematical Taxonomy of Institutional Constraints

Operational requirements are mathematically split into rigid Hard Constraints and flexible Soft Constraints for assessing the viability of the candidate timetable.

Hard Constraints

Hard constraints define non-negotiable physical and logical boundaries. Any violation invalidates the chromosome's structural viability, represented as an absolute penalty within the fitness function:

$$\sum \text{violations}_{\text{Hard}} = 0 \quad [2.2.]$$

1. **Lecturer Conflict Avoidance:** An individual lecturer (L_i) cannot be assigned to multiple concurrent courses within a single time slot (T_k).
2. **Space Double-Booking:** A specific physical venue (R_i) cannot host multiple class groups simultaneously.
3. **Student Cohort Overlaps:** A unique student batch (S_m) cannot have overlapping lecture modules assigned within the same time block.
4. **Resource Attribute Matching:** Modules requiring specialized equipment must match resource tags, such as mapping programming sessions to computer laboratories rather than traditional lecture halls.
5. **Availability Profile Matching:** The solver must completely bypass scheduling sessions during blocks flagged as unavailable on individual lecturer portals.

Soft Constraints

Soft constraints represent institutional preferences that optimize the academic experience without invalidating the schedule. The core engine minimizes these parameters to optimize operational comfort:

$$\text{Minimize } \sum \text{Penalties}_{\text{Soft}} \quad [2.3.]$$

1. **Prevention of Isolated Gaps:** Eliminating idle, single-period gaps within a student batch's daily timeline to ensure consecutive classes.
2. **Consecutive Lecture Thresholds:** Capping maximum continuous lecturing hours for instructors and student cohorts to mitigate fatigue.
3. **Mandatory Midday Intervals:** Enforcing structural lunch breaks within midday temporal blocks across active departments.
4. **Workload Balancing:** Evenly distributing the weekly course load across working days to avoid dense cluster days.

2.3. Research Gaps

A systematic analysis of contemporary literature reveals three fundamental research gaps that limit the practical deployment of automated scheduling systems in modern University environments:

Table 1: Summary of Identified Literature Research Gaps

Research Gap ID	Domain Category	Description of Institutional Limitation
GAP 1	Multi-Domain Isolation	Disconnected silos separating academic timetables, transit fleets, and event space bookings.
GAP 2	Dynamic Adaptability	Lack of real-time constraint interceptors for live administrative edits during mid-semester changes.
GAP 3	Curriculum Scaling	Database deadlocks and performance degradation in highly-coupled, shared-module degree structures.

2.3.1. Disconnected Multi-Domain Silos in Campus Resource Management

Existing metaheuristic timetabling systems work in isolation. They optimize academic classroom schedules without considering interconnected operations, such as university vehicle fleet allocations, and public event venues bookings. In reality, these domains share common physical infrastructure and administrative resources. Managing them through standalone software applications leads to uncoordinated cross-departmental conflicts, resource bottlenecks and excessive manual coordination. There is a clear absence of unified platforms capable of co-scheduling of academic, transportation, and venue resources within a single integrated framework.

2.3.2. Real-Time Dynamic Adaptability Deficits

Traditional metaheuristic algorithms operate as offline, batch-oriented solvers that generate fixed timetable documents prior to the start of an academic term. However, university operations are highly fluid and subject to daily disruptions, such as emergency faculty leave, sudden room maintenance, or ad-hoc event bookings. When a manual override is attempted mid-semester, traditional systems offer no live constraint validation. This creates a major gap: the lack of intelligent mechanisms capable of intercepting manual modifications, evaluating constraint violations instantly, and recommending feasible alternatives on-the-fly. UniSchedule addresses this gap through its proposed AI Conflict Interceptor.

2.3.3. Computational Scaling Limitations in Highly-Coupled Curriculum Networks

As degree curricula become increasingly interdisciplinary, universities frequently introduce **common or shared core subjects** that are taught concurrently to student cohorts from different departments and faculties. In traditional relational databases schemas and GA representations, modelling these shared subjects lead significant data redundancy, inefficient relationship modelling, and computational overhead during optimization. Existing literature lacks an optimized relational schema normalized to Third Normal

Form (3NF), together with an object-oriented GA framework capable of effectively scaling across highly coupled and complex curriculum networks without performance bottlenecks.

2.4. Comparative Analysis of Existing Approaches vs. UniSchedule

To justify the methodology and architectural choices of the proposed **UniSchedule** system, **Table 2** presents a comparative analysis between traditional manual methods, generic campus ERPs, existing literature models, and the UniSchedule platform.

Table 2: Comparative Matrix of Resource Scheduling Approaches and Technologies

Operational Dimension	Manual / Spreadsheet Solutions	Generic Campus ERP Systems	Existing Metaheuristic Literature	Proposed UniSchedule Ecosystem
Optimization Method	Manual human compilation	Rule-based heuristics/SQL queries	Isolated GAs / Tabu Search	Hybrid DEAP Genetic Algorithm Core
Domain Coverage	Disconnected domain silos	Separate, unlinked modules	Academic classroom timetabling only	Unified (Academic + Vehicle + Events)
Real-Time Conflict Handling	Manual phone/email coordination	Manual override without validation	Offline batch regeneration required	Online AI Conflict Interceptor
Demand Forecasting	None (Reactive booking)	Historical reporting only	Absent in timetabling models	Multi-Output Random Forest Regressor
System Architecture	Desktop local files	Costly, monolithic on-premise ERP	Standalone CLI / Academic scripts	Decoupled (React.js + FastAPI + MySQL)
Database Schema	Unstructured / Duplicate records	Rigid, complex relational tables	In-memory arrays / Flat files	Normalized 3NF + Junction module_degrees
User Access Model	Single-user file access	Basic role permissions	Non-interactive execution	Multi-Portal RBAC (5 Secure Roles)

2.5. Summary of Key Findings from Literature

The literature and comparative analysis create the strategic roadmap for the UniSchedule system:

- **Hybrid Metaheuristic Efficiency:** Evolutionary GA tools developed using Distributed Evolutionary Algorithms in Python (DEAP) evolutionary computation framework provide a robust approach for solving NP-hard timetabling problems, provided that hard constraints are given higher priority than soft constraints penalties.
- **Decoupled Asynchronous Architecture:** Separating the presentation layer (React.js) from the asynchronous API server (FastAPI) and connection-pooled relational database (MySQL via SQLAlchemy) enables efficient background optimization while maintaining a responsive user interface.
- **Proactive Live Interception:** An Artificial Intelligence (AI) Conflict Interceptor can intercept manual administrative modifications in real time and validate them against predefined scheduling constraints, thereby, preventing cascading cross-departmental conflicts and maintain schedule integrity throughout the academic term.
- **Predictive Machine Learning Integration:** By analyzing historical transit logs, Random Forest regression models can accurately forecast future demand for university vehicle fleets and anticipate potential resource shortages, bridging the gap between static academic scheduling and dynamic auxiliary operations.

The UniSchedule is a secure, multi-portal web application that integrates metaheuristic optimization, real-time AI conflict interception, a normalized 3NF relational schema design, RBAC and predictive ML to directly address the identified research gaps and provide a scalable resource management framework for modern universities.

3. Materials and Methods

3.1. Overview and System Architecture

To resolve the operational challenges of resource scheduling and transport demand forecasting, a decoupled, multi-tier web application architecture was implemented. As illustrated in **Figure 1**, the system separates user-facing frontend interfaces from high-performance scheduling routines, machine learning engines, and data persistence layers, enabling scalable, real-time constraint validation.

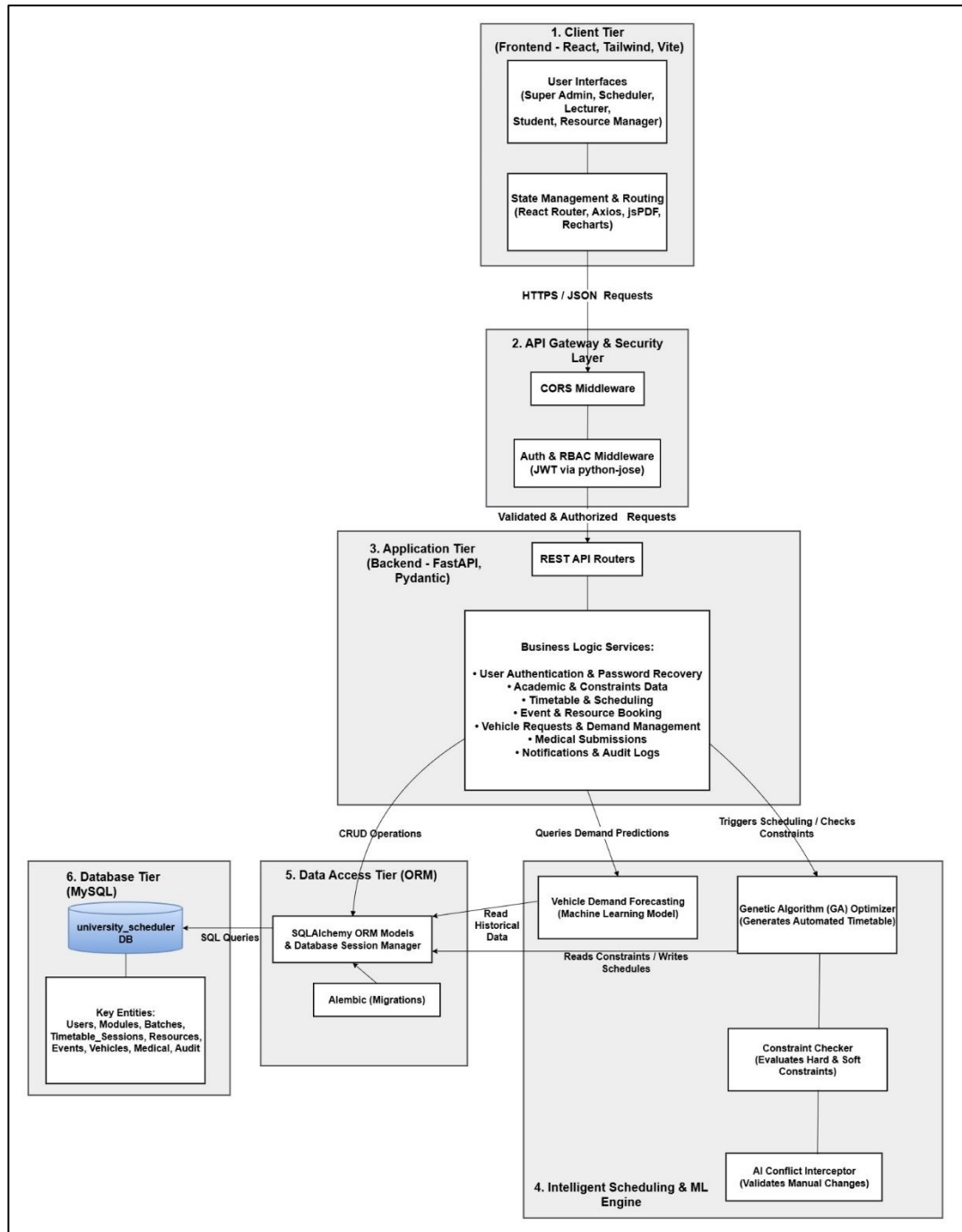


Figure 1: High-Level System Architecture

As depicted in **Figure 1**, the system architecture spans six core functional layers:

1. **Client Tier (Frontend):** Implemented using **React (v18)**, **Tailwind CSS**, and **Vite**. It delivers responsive, role-tailored user interfaces for Super Admins, Schedulers, Lecturers, Students, and Resource Managers. Client-side routing and HyperText Transfer Protocol (HTTP) communication are managed via React Router and Axios.
2. **API Gateway & Security Layer:** Configured with Cross-Origin Resource Sharing (CORS) middleware to regulate origin requests and Role-Based Access Control (RBAC) enforced via JSON Web Tokens.
3. **Core Application Tier (Backend):** Powered by FastAPI, handling core business routines including academic module management, booking requests, medical submissions, and audit tracking.
4. **Intelligent Scheduling & ML Engine:** Contains the computational engines: the Genetic Algorithm (GA) timetable generator (built on the DEAP library), real-time Constraint Checker, and Vehicle Demand Forecasting Engine.
5. **Data Access Tier:** Uses SQLAlchemy Object-Relational Mapping (ORM) for query generation and schema abstraction, combined with Alembic for incremental database migrations.
6. **Database Tier:** A relational MySQL database providing persistence for system users, timetable records, venue constraints, and vehicle logs.

The experimental and development environment specifications used to implement and test the platform are detailed in **Table 3**.

Table 3: Experimental and Development Environment Specifications

Layer / Component	Technology / Library	Description & Function
Frontend UI	React, Vite, Tailwind CSS	Responsive SPA frontend execution
State & API Gateway	Axios, React Router, Recharts	Async HTTP handling & analytics rendering
Backend Framework	FastAPI (Python 3.10+)	High-performance asynchronous REST API server
Optimization Engine	DEAP Framework (Python)	Distributed Evolutionary Algorithms for timetable generation
Machine Learning	Scikit-Learn, Pandas	Regression models & feature engineering for demand forecasting
Database & ORM	MySQL, SQLAlchemy, Alembic	Relational storage & database migration tracking
Security Protocol	JWT, bcrypt	Stateful RBAC & encrypted token verification

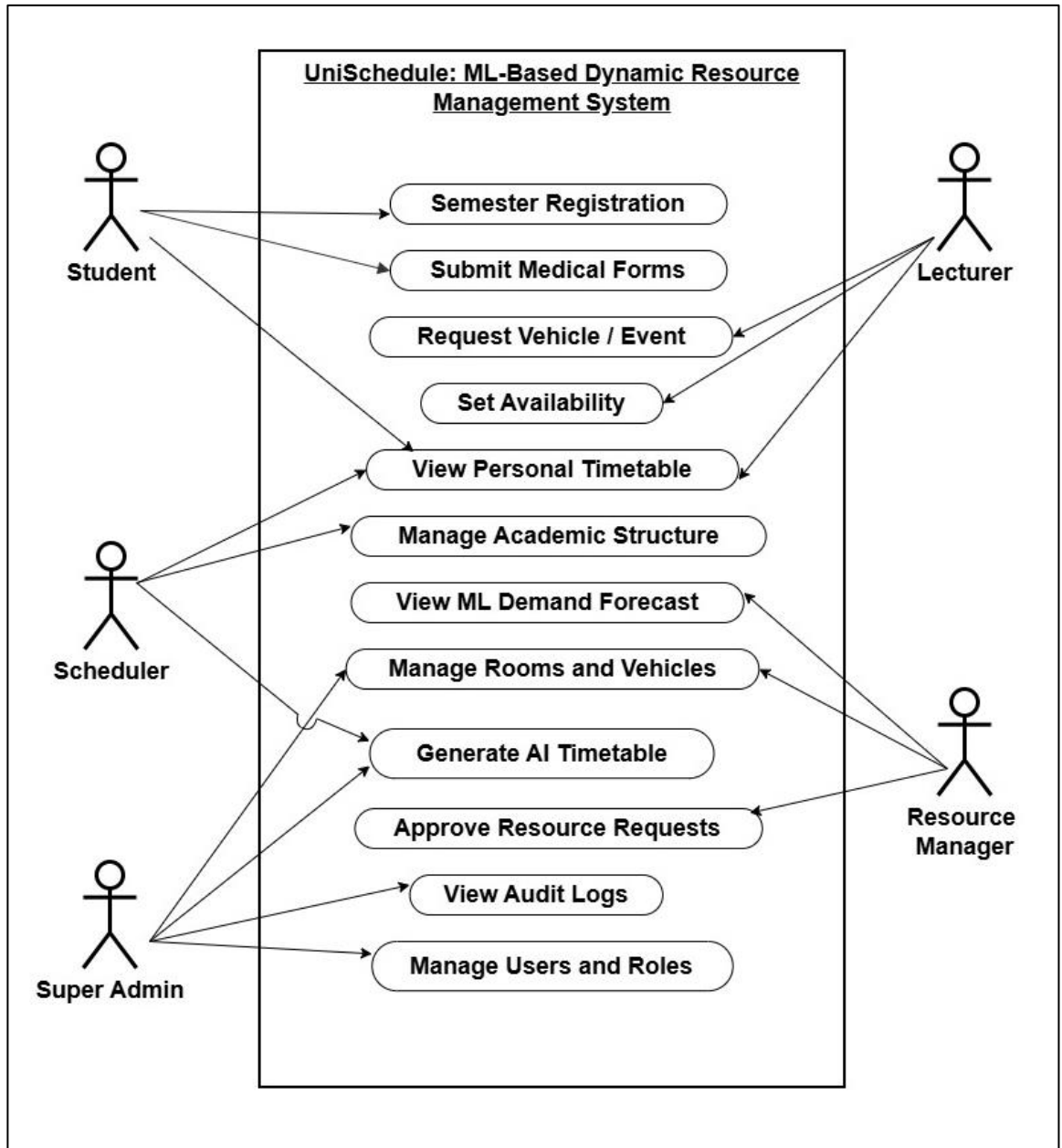


Figure 2: Use Case Diagram of the UniSchedule System

3.2. Mathematical Formulation of Timetabling Constraints

The automated scheduling problem is modelled as a Constraint Satisfaction and Optimization Problem (CSOP). Let:

$T = \{t_1, t_2, \dots, t_N\}$ be the set of available weekly time slots.

$R = \{r_1, r_2, \dots, r_K\}$ be the set of physical lecture halls/labs.

$L = \{l_1, l_2, \dots, l_M\}$ be the set of academic lecturers.

$B = \{b_1, b_2, \dots, b_P\}$ be the set of student batches.

$M = \{m_1, m_2, \dots, m_Q\}$ be the set of academic modules.

Soft Constraints (Optimization Criteria)

1. **Lecturer Preferred Time Slots:** Minimizes sessions scheduled during non-preferred hours.
2. **Batch Idle Time Minimization:** Minimizes unallocated gaps between consecutive lectures on the same day for a batch.

3.3. GA Engine Implementation

The automated timetable generation process is based on an evolutionary optimization workflow using the GA as shown in the flowchart below in **Figure 3**. Firstly, it fetches input data, which consists of loading the modules, lecturers to be assigned, target student batches, physical room locations, and system scheduling constraints. The initial population, or starting population, is then randomly generated and made up of candidate schedules of N individuals. Fitness evaluation is performed on each schedule in the population, using a specific function for constraint checking, which assigns penalties depending on the constraint satisfaction: the penalty values are higher for hard constraints (e.g. room double booking, lecturer double booking) than for soft constraints (e.g. preferred working hours, minimum gap between two schedules). After fitness scoring, it tests if any termination condition is fulfilled, which in this case are the following: maximum generation reached or optimal fitness reached (100%).

If the termination condition is met (Yes), the algorithm chooses the best candidate schedule, writes the last optimized schedule to the Timetable Sessions database table and stops the process. In the opposite case, however, when the termination condition is not fulfilled (No), the evolutionary cycle is continued: Best schedules (parents) are selected by a selection operator and a crossover operator combines structural segments of two parent schedules to create offspring. To prevent the formation of local optima a small percentage of the session allocations are randomly changed by a mutation operator, e.g., by changing timeslots or room assignments. Finally, the old population is replaced by the mutated offspring and the iterative cycle is repeated to determine the fitness of the new population of schedules until an optimum timetable is found.

Genetic Algorithm (GA) Process Flow for Automated Timetable Generation

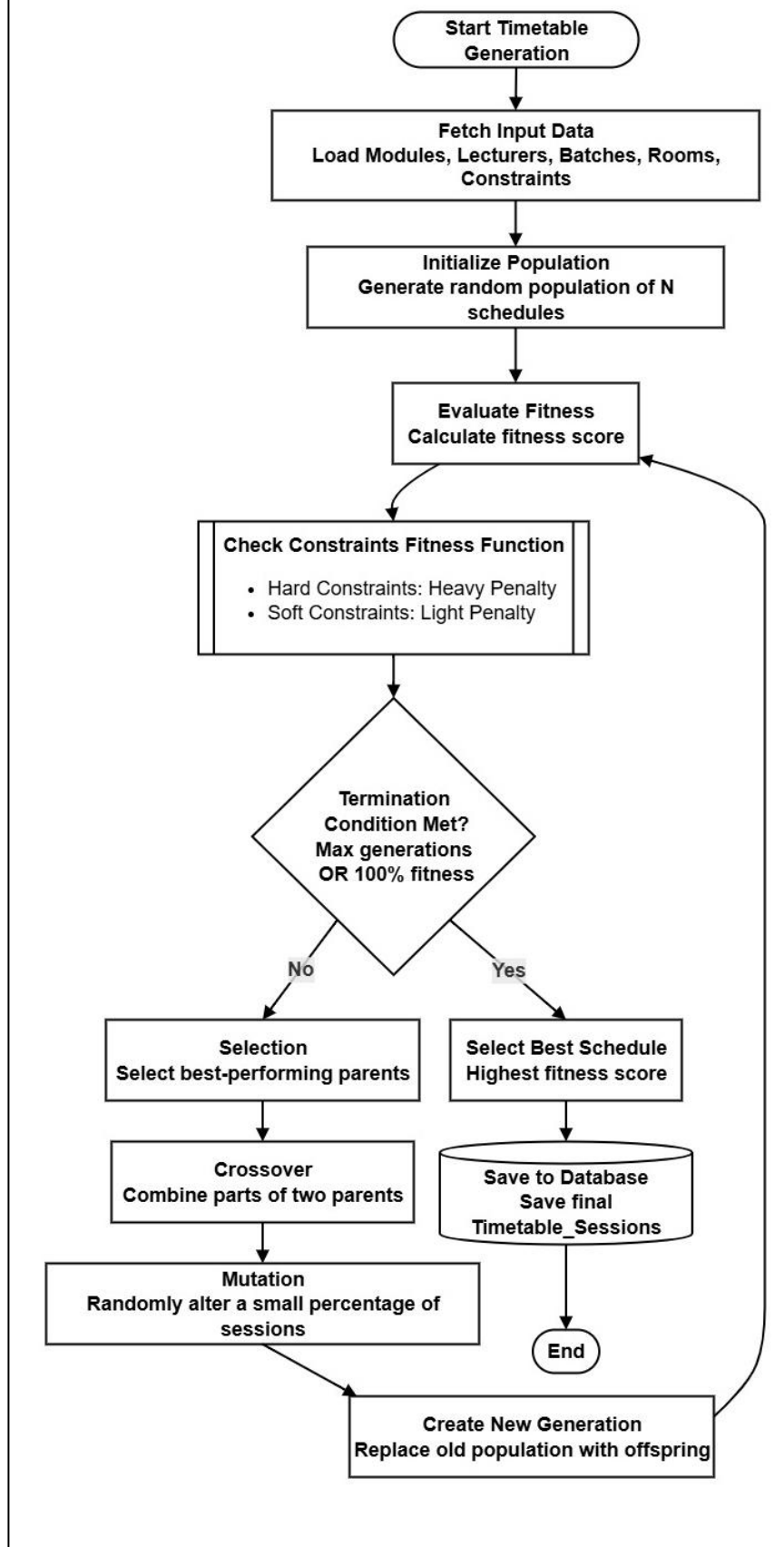


Figure 3: Genetic Algorithm (GA) Process Flow for Automated Timetable Generation

3.4. Automated Conflict Resolution and AI Suggestion Mechanism

When an administrator attempts a manual schedule change, the system executes real-time validation across all active hard and soft constraints.

The system uses an automated Artificial Intelligence (AI) Conflict Interceptor, as outlined in **Figure 4**, to ensure schedule integrity when there are ad-hoc requests or manual administrative changes. Once the User Request is received the process is initiated and all the parameters received are parsed and validated in sequence through a sequential decision tree:

- **Check Resource Availability:** This function makes sure that the room or vehicle is not already booked at that specified time.
- **Lecturer Availability Check:** If the resource is available (Yes), then the interceptor checks for any overlapping class with the requested lecturer or if the lecturer is marked as unavailable.
- **Student Batch Availability Check:** If the lecturer is available (Yes), it will determine whether there is already another module available for the same batch of students.

If none of the three validation checks fail (Yes), the request is processed without conflict, the database is updated with the new record and the execution flow continues until the end of the flow.

On the other hand, if any condition is not met (No) at any point, then the interceptor immediately branches to the Identify Conflict Source module to log the specific failed condition (e.g. Room 401 is booked for IT1010). This activates the AI Suggestion Module, which will query the Genetic Algorithm engine and database to calculate the next best time slots/alternate rooms available to satisfy the request constraints.

Then the system performs the step Alert User & Show Alternatives that shows the conflict error message and a list of slots that the AI recommends as alternatives, directly on the user interface. Lastly, the loop for user acceptance of suggestion is checked:

- When the user clicks on Yes, Use Suggestions, the system runs Approve & Update to store the alternative allocation in the database.
- Upon clicking on No/Cancel, Discard Changes is performed, which means that the manual editing is cancelled and the current schedule is left untouched.

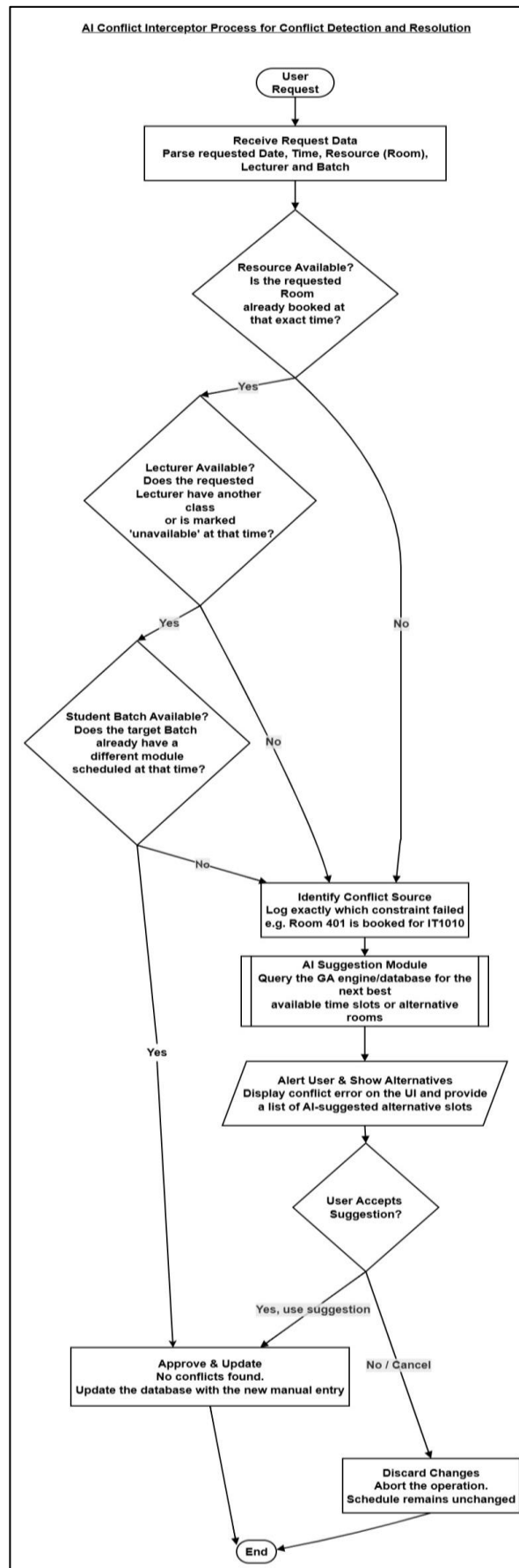


Figure 4: AI Conflict Interceptor Process for Conflict Detection and Resolution

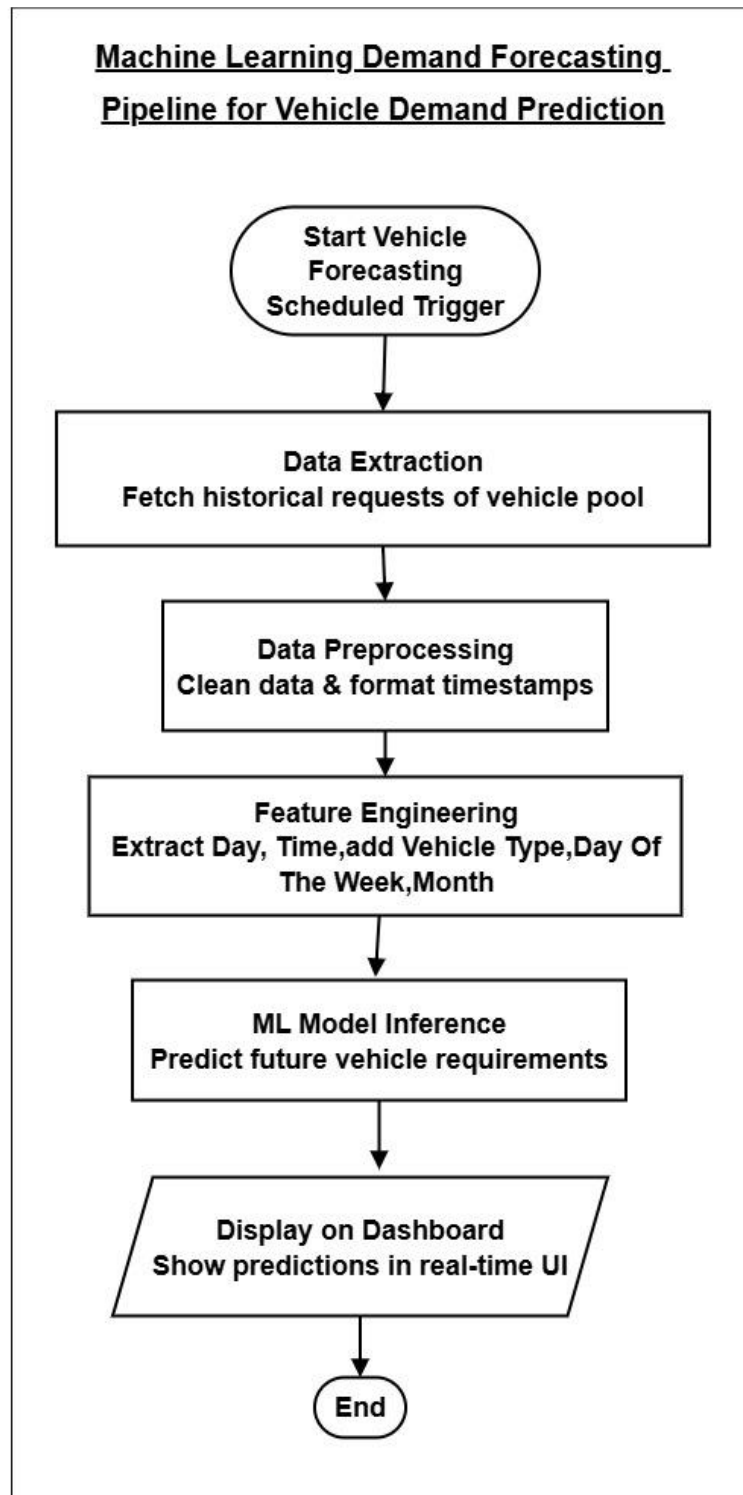


Figure 5 : Machine Learning Demand Forecasting Pipeline
for Vehicle Demand Prediction

As shown in **Figures 4** and **5**, when a manual edit violates a constraint:

1. The engine catches the specific failure point (e.g., Lecturer overlapping).
2. The **AI Suggestion Module** scans nearest open slots matching the lecturer and batch availability.
3. Recommended options are returned to the React frontend.
4. Accepting a suggestion applies the update to the MySQL database; declining cancels the edit.

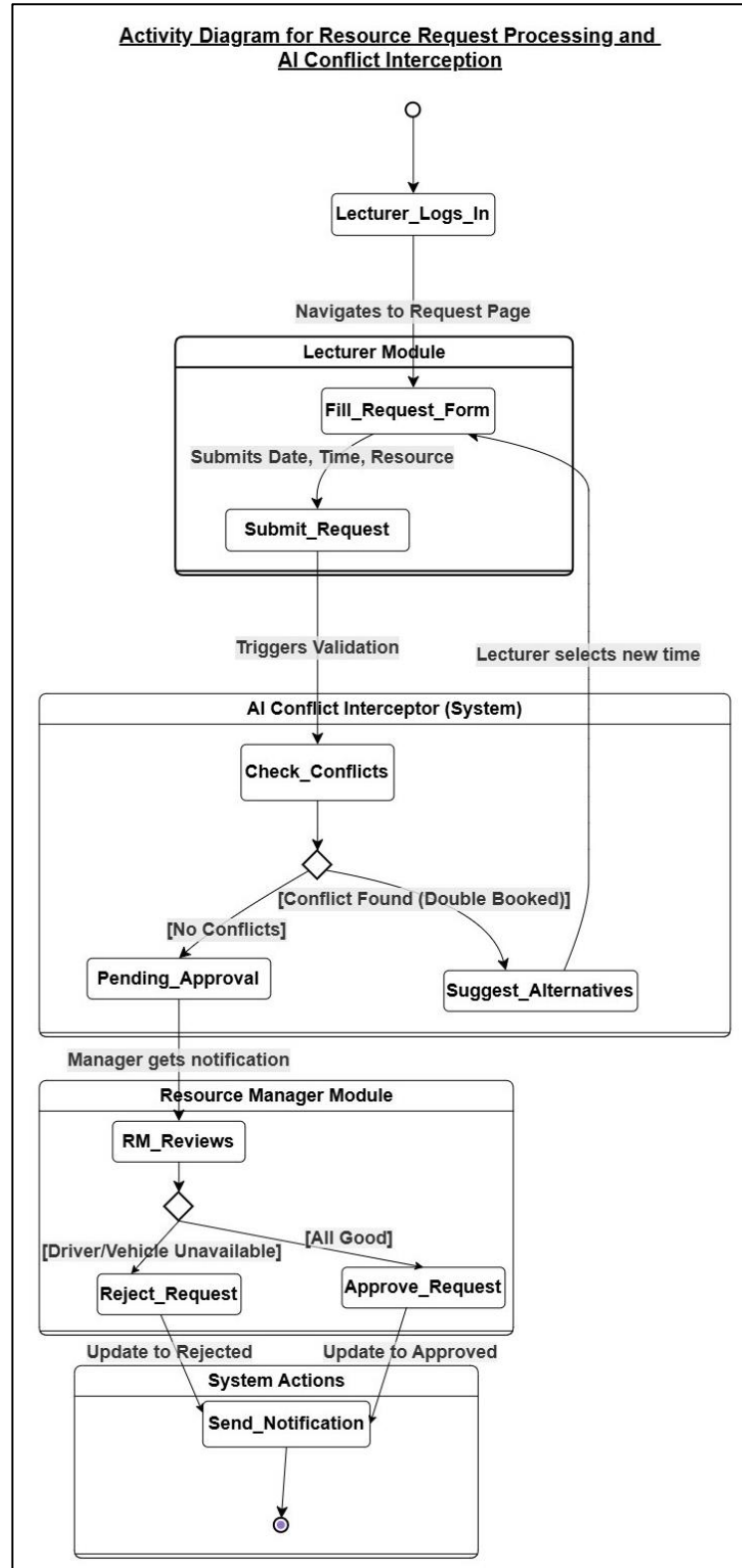


Figure 6: Activity Diagram for Resource Request Processing and AI Conflict Interception

To further illustrate the end-to-end user interaction and system validation process, **Figure 6** presents the Activity Diagram for Resource Request Processing. As illustrated in the workflow, when a user (such as a lecturer) submits a request for a physical resource or time slot, the system immediately activates the AI

Conflict Interceptor. The interceptor evaluates the request against existing scheduling constraints and resource availability stored in the database. If a conflict is detected, the system prevents the request from being submitted and dynamically suggests suitable alternative options. If no conflicts are identified, the request is successfully forwarded to the Pending state, where the Resource Manager reviews and either approves or rejects the request, ensuring a streamlined and conflict-free resource allocation process.

3.5. Machine Learning Vehicle Demand Forecasting

To prevent transport shortages during academic field activities, a scheduled predictive pipeline estimates upcoming vehicle demand.

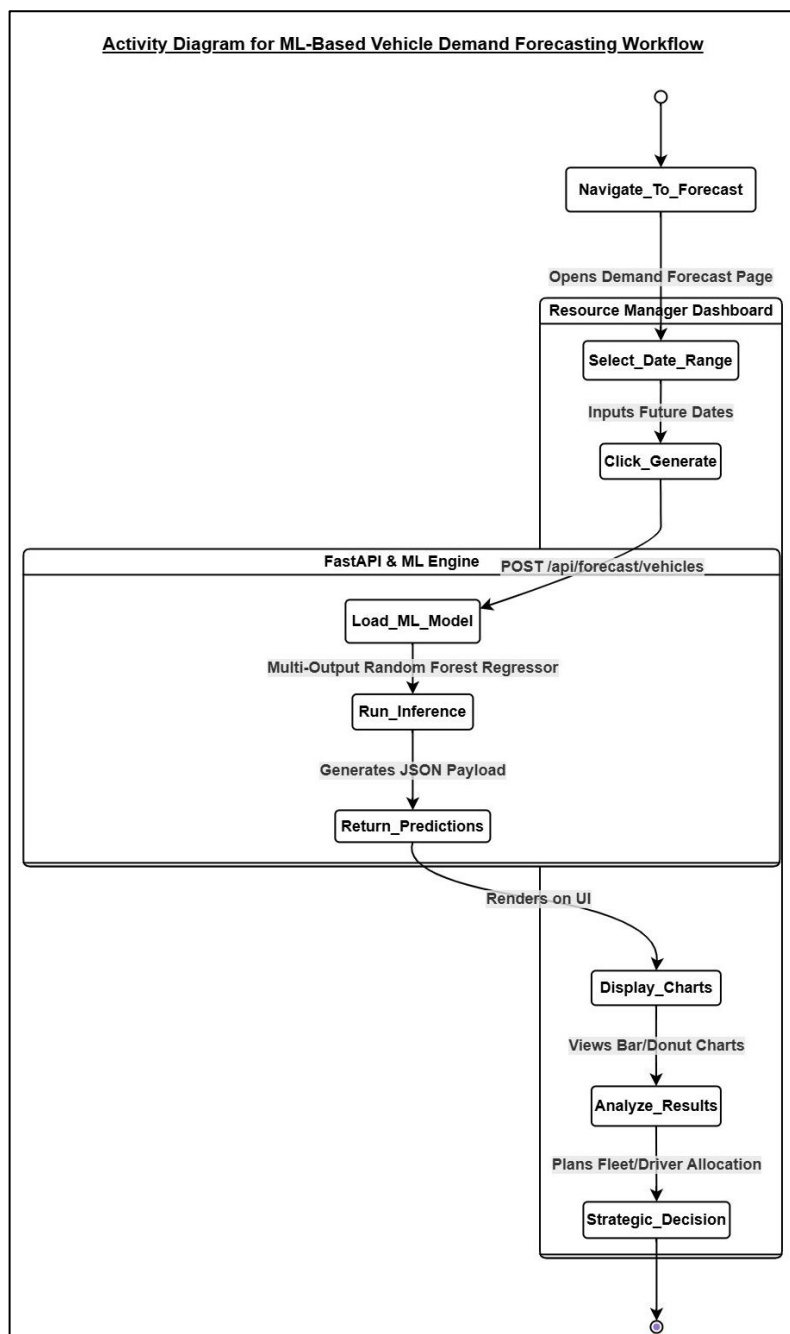


Figure 7: Activity Diagram for ML-Based Vehicle Demand Forecasting Workflow

3.5.1. Feature Engineering and Pipeline Sequence

1. **Data Extraction & Preprocessing:** System logs, event timetables, and past vehicle requests are extracted and cleaned. Categorical features are encoded, and timestamp features are converted to numerical formats.
2. **Feature Engineering:** Key features are extracted, including `day_of_week`, `time_slot`, `academic_week`, `batch_size`, and rolling historical request averages.
3. **Inference & Capacity Verification:** The ML model (Scikit-Learn) predicts incoming transport requirements.
4. **Automated Alerting:** If predicted vehicle demand exceeds total fleet capacity (Yes), an alert is sent to the Resource Manager's dashboard. Predicted metrics are rendered on the UI.

3.6. Performance and Evaluation Metrics

To evaluate system efficiency and predictive performance, three evaluation standard metrics were defined:

1. **Scheduling Optimization Efficiency:**
 - **Execution Time (s):** Computational overhead for GA convergence across variable input sizes.
 - **Constraint Satisfaction Rate (CSR):**

$$CSR = \left[\frac{\text{Satisfied Soft Constraints}}{\text{Total Soft Constraints Evaluated}} \right] \times 100\% \quad [3.1.]$$

2. **System API Responsiveness:**

Measured in milliseconds (ms) using Locust load testing scripts across core endpoints.

4. Technology Consideration

The system is built with a decoupled, responsive and modern full stack web architecture. This separation of the user interface and processing/optimization logic, which places the user interface (frontend Client) and the data processing/optimization logic (backend API) on two different servers, allows for heavy computational requirements (evolutionary optimization algorithms, etc.) to have no impact on the user interface, preventing lag or browser timeouts.

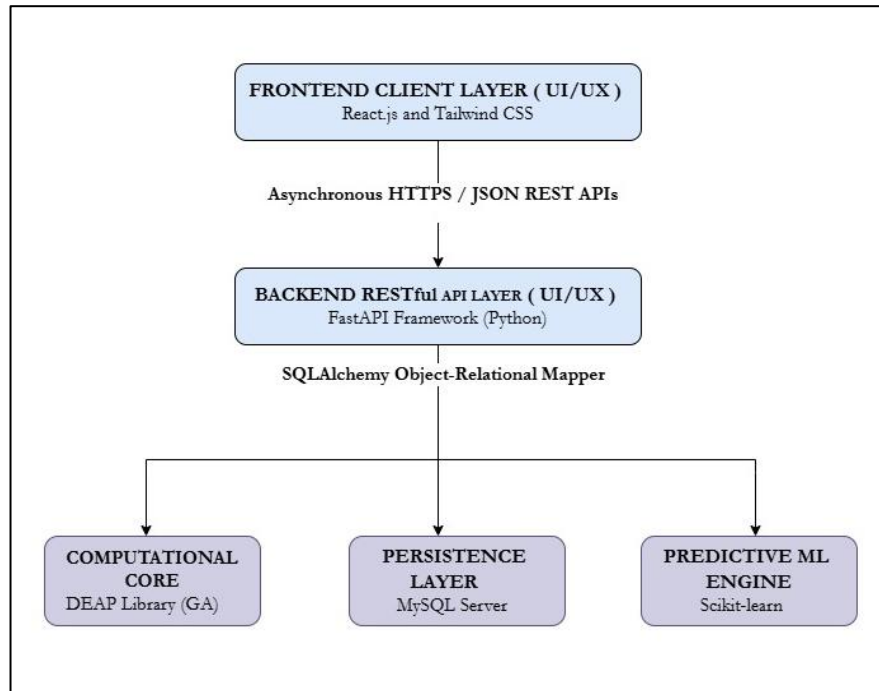


Figure 8: Technological Layers

4.1. Frontend Client Layer Architecture

The client application is a Single Page Application (SPA) that is based on the React.js component architecture and styled with the Tailwind CSS framework.

4.1.1. React.js and Virtual DOM Optimization

Schedules are traditionally displayed on the Web page as HTML tables. If many table cells (e.g., time slots, modules, rooms, batches) are changed at the same time by user interaction (e.g., drag-and-drop), the browser has to recompute the geometry of the entire page. This results in a lot of rendering lag and visual stutter.

The Virtual DOM rendering pipeline is how React.js addresses this structural performance problem:

- **In-Memory Diffing:** When an administrative user moves a lecture block, the application creates a lightweight in-memory snapshot of the updated interface.

- **Selective Re-rendering:** React's reconciliation algorithm compares the new Virtual Document Object Model (DOM) with the previous state, isolates the exact table cells that changed, and batches the update to the real browser DOM in a single operation. This ensures fluid, zero-lag performance even when displaying dense, multi-faculty academic matrices.

4.1.2. Responsive Utility-First Presentation with Tailwind CSS

Typically, university administrators have wide desktop monitors while lecturers and students are much more dependent on mobile smartphones and tablets for real time updates. Tailwind CSS is a mobile first, utility first layout engine that can resolve this interface design problem:

- **Cleaner frontend application package size:** The pre-compiled, atomic CSS utility classes are specified directly in the React layout components, which means that the size of the frontend application is significantly reduced, and load times will be fast even on a slow campus Wi-Fi network.
- **Fluid Grid Adaptability:** The large timetable grid will collapse to clean, swipeable single column cards on small screens on mobile devices. It completely avoids visual clutter, text overlapping and broken interfaces without having to download separate mobile applications.

4.2. Backend RESTful API Layer Architecture

The back end is written entirely in Python using the state-of-the-art FastAPI ecosystem. It is the central orchestrator that is responsible for handling data validation, secure authentication and execution triggers for the optimization core.

4.2.1. Asynchronous Event-Driven Concurrency

FastAPI is designed to be built natively on top of Asynchronous Server Gateway Interface (ASGI) and the Starlette framework, which is a different synchronous Python framework from Django or Flask, treating a single thread of a computer for every network request.

- **Non-blocking I/O Operations:** If a student logs in to the dashboard, or the system generates automated registration forms, the backend thread will not wait in line for the database or disk to respond. Rather, the thread is liberated asynchronously (async/await) so that it can handle the other network requests.
- **High-Concurrency Readiness:** This architecture enables a small server to easily manage high-concurrency environments like hundreds of students looking at their customized lecture schedule at the same time during rush hour in the mornings without losing connections or crashing.

4.2.2. Strict Data Validation with Pydantic

One of the most important things you have to do is guarantee that your data will not contain typos or missing data before it gets to the optimization algorithm. Data typing is strict in FastAPI, with the help of Pydantic data models:

- **Runtime Data Scrubbing:** All the data sent to the backend (such as a CSV file with course listings or a JSON payload with lecturer availability metrics) is intercepted by the API gate.
- **Automatic Error Interception:** Pydantic automatically parses, sanitizes, and validates the attributes being passed in against strict schemas defined by it (e.g., room capacities must be positive integers). In the event of invalid data, the API responds straight away with a JSON error message without corrupting any records on the database.

4.3. Persistence and Object-Relational Mapping (ORM) Layer

A centralized MySQL Relational Database is used for data persistence with SQLAlchemy Object-Relational Mapper (ORM) as the link between the backend code and the database.

4.3.1. Relational Integrity and Junction Engineering

The nature of a university schedule is such that it's composed of very interconnected real-world entities, so that a relational database structure is a must. For instance, a single Lecturer Allocation record needs to reference directly to a particular User ID, Course Code and Room ID. Foreign key constraints in MySQL are set up strictly, thus avoiding orphan data entries.

Additionally, the database design includes a optimized Many-to-Many junction table schema (module_degrees) that is used to represent common curriculum modules (such as basic math courses used by multiple degrees). This prevents data duplication, so when a course title is changed, it shows up immediately on any degree path it's connected to, without causing mismatches in the database.

4.3.2. SQLAlchemy Mapping and Connection Pooling

SQLAlchemy serves as a programmatic bridge, abstracting database rows into native Python objects:

- **Query Security:** By utilizing the SQLAlchemy Expression Language, the system is natively protected against cross-site scripting and SQL injection attacks.
- **Connection-Pooling Configurations:** To prevent database locks during intensive data modifications, the MySQL engine is configured with connection-pooling properties:

Pool Size = 5, Max Overflow = 10, Pool Timeout = 30 seconds

With this setup, the background Genetic Algorithm engine writes thousands of new timetable records to the database with no interruption to normal use of the system by the students (e.g., a student reads a notification).

4.4. Computational Optimization Core

The core optimization, automatic timetable compilation features are powered by the DEAP (Distributed Evolutionary Algorithms in Python) library.

4.4.1. Object-Oriented Genetic Programming

DEAP has a mathematically well-established foundation for its Genetic Algorithm, so there is no need to write the entire thing from scratch, which results in a "messy" and "not optimized" code. It encodes the timetabling problem as specific genetic concepts:

- **Individual (Chromosome):** Modelled as a multi-dimensional array representing a complete weekly schedule configuration. Each individual slot contains specific parameters:

***Slot* = {Time Slot, Day, Room, Lecturer, Student Batch} [4.1.]**

- **Population:** A collection of these individual schedules (typically configured to a size of 50 to 500) that compete against each other based on their overall efficiency score.

4.4.2. Evolutionary Operator Customization

The DEAP computational module is a multi-threaded service, directly embedded inside the FastAPI backend. It enables the development team to set very specialized evolutionary operators:

- **Tournament Selection:** Evaluates the fitness of small random groups of schedules, picking the best ones to move into the next generation.
- **Two-Point Crossover:** Swaps large logical blocks of schedules (such as swapping an entire day's classroom assignments between two parent timelines) to combine successful configurations.
- **Adaptive Mutation:** Randomly shifts single classes to completely new rooms or times with a small probability (typically $p = 0.05$). This injects new scheduling combinations and prevents the algorithm from getting stuck in local optimization traps.

By combining the DEAP computational engine with an asynchronous backend and a responsive frontend, the system delivers an end-to-end, production-ready smart scheduling application.

4.5. Predictive Analytics and Machine Learning Engine

To support the dynamic vehicle booking infrastructure, the platform integrates an intelligent predictive demand forecasting module built using the core Python data science ecosystem.

4.5.1. High-Performance Data Manipulation with Pandas

Before feeding raw university data into the predictive model, the system utilizes the **pandas** library to handle large-scale data manipulation and feature engineering. Historical booking logs, transit requests, seasonal academic calendars, and departmental transit records are structured into vectorized data frames. Pandas is selected for its high-performance computational efficiency, allowing the system to handle missing data fields, perform temporal aggregation (grouping data by week, semester, or exam seasons), and transform categorical variables seamlessly without introducing memory bottlenecks to the backend environment.

4.5.2. Robust Predictive Modelling with Scikit-learn

The predictive core is engineered using the **scikit-learn** framework, deploying a Random Forest regression pipeline. Scikit-learn provides an optimized, reliable execution pipeline for ensemble learning methods, which are less prone to overfitting compared to single decision trees. The Random Forest model processes the pandas-engineered features to execute **predictive demand forecasting** for the university vehicle fleet. By analyzing historical multi-variable datasets, the scikit-learn engine outputs highly precise utilization forecasts, predicting future transportation demand peaks and potential fleet shortages weeks in advance. This predictive intelligence allows the backend to proactively alert asset managers and optimize fleet distributions before resource allocation conflicts take place.

4.6. Cloud Infrastructure and Deployment

The UniSchedule platform is cloud-ready and uses Amazon Web Services (AWS) in conjunction with dedicated third party APIs to deliver high availability, scalable persistence and efficient media management.

4.6.1. Cloud Infrastructure and Services Breakdown

- Amazon Elastic Compute Cloud (EC2) with Elastic IP: Amazon elastic compute cloud instance running Ubuntu with Elastic IP to host the containerized frontend (React.js) and the backend (FastAPI). A dedicated Elastic IP is attached to the instance to ensure that there is a fixed, reliable IPv4 address for the instance to communicate with other clients on the Internet, even when the instance restarts.

- Amazon Relational Database Service (RDS) for MySQL: Acts as the main persistence layer. RDS securely manages the multi-tenant relational database schema, handles SQLAlchemy connection pools needed for the background Genetic Algorithm, and provides automated daily backups to avoid institutional data loss.
- Rather than storing, managing and accessing user-uploaded files on the local server or using traditional object storage buckets, Cloudinary seamlessly pulls in the external Cloudinary API for user file uploads such as student medical certificates and dynamically generated PDF timetables, safely stored, managed and retrieved. This integration helps minimize the amount of data the primary EC2 instance is required to store, as well as reduce asset delivery speeds.

4.6.2. Key Advantages of the Deployment Architecture

- Efficient Resource Usage: The EC2 instance's primary computational resources can now be used exclusively for the DEAP evolutionary algorithms and asynchronous requests by the FastAPI server, thanks to the offloading of media storage to the Cloudinary CDN.
- Data Reliability and Security: Amazon RDS provides strong relational data security features including encrypted connections and snapshot backups, which allows for critical university scheduling data to be kept safe and secure.

5. Development, Testing, Implementation and Evaluation

5.1. System Development Workflows and Feature Implementation

The UniSchedule platform was developed by an iterative process of Design Science Research (DSR), which involved integrating user-friendly interfaces, asynchronous RESTful Application logic, Relational Database support for data persistence, and the Computational Intelligence engines. It was realized as a modular web application communicating with a stateless back-end API gateway.

Table 4: High-Level Full-Stack System Topology and Tier Interactions

UNISCHEDULE SYSTEM TOPOLOGY	
CLIENT TIER	React.js + Tailwind CSS + Axios + Recharts
API GATEWAY	FastAPI + CORS Middleware + JWT Authentication
COMPUTATIONAL	DEAP Evolutionary Algorithm + Scikit-Learn Random Forest Pipeline
PERSISTENCE	SQLAlchemy ORM + Alembic Migrations + MySQL Engine

5.1.1. Full-Stack System Architecture and Modular Component Breakdown

The application layer was designed as separate functional parts for maintainability and separation of concerns. As shown in **Table 4**, the main technical layers in the UniSchedule ecosystem are interacting in the following ways:

- **Frontend Presentation Layer:** React.js and styled with Tailwind CSS. The Axios library is used for state management, to communicate with the HTTP API asynchronously, and Recharts are used for data visualization components. Axios is used for data state management and to communicate with the HTTP API asynchronously, Recharts are used for data visualization.
- **API Gateway (backend):** Designed using Python and FastAPI. It is automated to sanitize the input using Pydantic schemas, and provides OAuth2 password-bearer authentication.
- **Algorithmic Core:** Contains the Genetic Algorithm (GA) timetable generator, which uses the DEAP library, an online AI Conflict Interceptor and the Multi-Output Random Forest regression model for vehicle demand forecasting.

5.1.2. Relational Database Normalization and Junction Schema Engineering

The relational persistence layer was designed and normalized to Third Normal Form (3NF) using MySQL 8.0. One of the main schema design challenge was modelling common or shared topics (courses offered to multiple groups of students across different faculties and degree programmes).

To avoid this data duplication, a junction entity relationship was created between modules and degrees by designing an optimized junction schema (`module_degrees`), which is a Many-to-Many relationship. Enforces relational integrity with foreign keys with cascading actions, database index optimisation on often queried foreign key fields (`batch_id`, `faculty_id`, `tenant_id`).

5.1.3. Role-Based Access Control (RBAC) and User Portals

A role-based access control (RBAC) system is used to control user authorization, with JSON Web Tokens (JWT) using a secret key cryptography signing system. The `bcrypt` algorithm is used to generate cryptographic hashes of user passwords.

The application provides five different portals customized for users:

- **Administrator Portal:** Manages complete Master Data Management (CRUD operations) for Users, Faculties, Departments, Degree Programs, Student Batches, Lecture Halls, and Vehicle Inventories, while configuring academic constraints and global parameters such as maximum daily consecutive lecture hours and mandatory lunch breaks.
- **Scheduler Portal:** Oversees faculty-based timetable generation, resolves operational scheduling conflicts, and manages localized faculty schedules.
- **Lecturer Portal:** Contains a time-slot calendar for booking non-preferred/unavailable hours, for checking own module allocation and for booking venue and transport.
- **Student Portal:** Renders academic timetables, processes online medical form submissions with administrative tracking, enables students to generate and download semester registration forms, and provides centralized access to additional student support services.
- **Resource Manager Portal:** Manages venue and vehicle requests and tracks venue machine learning vehicle demand predictions.

5.2. Comprehensive Software Testing Framework and Results

The UniSchedule system was subjected to functional correctness, user interface responsiveness, database constraint correctness, and algorithmic correctness software testing to validate the system throughout all the user portals.

5.2.1. Overview of System Test Suite Execution

A structured test suite was used to test the 5 dedicated portal modules. The test execution was done by the assigned Quality Assurance (QA) testers.

For the first pass, the system obtained an initial pass rate of 88.00% and an initial defect rate of 10.22%. After the developers fixed the critical defects, the end result of the system pass rate was 100.00% verified.

5.2.2. Portal-Wise Test Execution Metrics and Analysis

Table 5: Empirical Portal-Wise Software Testing Audit Summary

Portal Module	Tested Role / Scope	Initial Pass Rate (%)	Final Pass Rate (%)	Status
Admin Portal	System Master Data & Users	90.55%	100.00%	All Defects Resolved & Passed
Lecturer Portal	Availability & Timetable	100.00%	100.00%	Verified & Passed
Resource Manager	Venues & Vehicle Fleet	85.71%	100.00%	All Defects Resolved & Passed
Scheduler Portal	Timetable Generation & Rules	81.25%	100.00%	All Defects Resolved & Passed
Student Portal	Timetable, Reg & Medicals	63.64%	100.00%	All Defects Resolved & Passed
TOTAL ECOSYSTEM	Full Application Scope	88.00%	100.00%	Ecosystem Validated

Table 5 presents the results of the defect resolution cycles for all five operational portals, which were Admin, Lecturer, Resource Manager, Scheduler and Student, with each achieving a final pass rate of 100.00%.

5.2.3. Representative Test Cases Across Functional Modules

To give a sense of the breadth of testing, Table 4 shows a subset of test cases taken directly from the system test logs for the five user portals.

AdminTable				
Test ID	Module	Steps to Test	Expected Result	Status (Pass/Fail)
TC-ADM-004	Dashboard	Access dashboard without login	Redirect to Login page	Pass
TC-ADM-005	Dashboard	Verify statistics cards	Correct counts displayed (have test)	Pass
TC-ADM-006	All Users	Open user list	User list displayed	Pass
TC-ADM-007	All Users	Search existing user	Correct user displayed	Pass
TC-ADM-008	All Users	Search non-existing user	No results message displayed	Pass
TC-ADM-009	All Users	View user details	User information displayed	Pass

Figure 9: Test Cases for the Admin Portal

LecturerTable					
Test ID	Module	Steps to Test	Expected Result	Status (Pass/Fail)	
TC-LEC-001	Login	Leave email and password empty and click Login.	Required field validation messages displayed.	Pass	
TC-LEC-002	Login	Enter invalid email and password and click Login.	Invalid credentials error message displayed.	Pass	
TC-LEC-003	Login	Enter valid email and invalid password and click Login	Invalid credentials error message displayed.	Pass	
TC-LEC-004	Login	Enter valid email and valid password and click Login.	User successfully redirected to Dashboard.	Pass	

Figure 10 : Test Cases for the Lecturer Portal

ResourceManagerTable					
Test ID	Module	Steps to Test	Expected Result	Status (Pass/Fail)	
TC-RES-001	Login	1. Navigate to the login page url. 2. Enter a valid university email (resource@uwu.ac.lk). 3. Enter the correct password. 4. Click the "Sign In" button.	The system authentication succeeds, and the user is securely redirected to the Resource Manager Dashboard page.	Pass	
TC-RES-002	Login	1. Enter an invalid email format or password. 2. Click "Sign In".	The system blocks the submission and displays an inline validation error message.	Pass	

Figure 11: Test Cases for the Resource Manager Portal

SchedulerTable					
Test ID	Module	Steps to Test	Expected Result	Status (Pass/Fail)	
TC-SCH-003	Timetable Generation	1. Navigate to Timetables > Generation. 2. Select a Faculty (e.g., FOT). 3. Attempt to click and select options from the Department and Degree Program dropdown menus under Target Selection.	The dropdown menus should dynamically populate and allow selection of specific departments and degree options belonging to the chosen faculty.	Fail	
TC-SCH-004	Timetable Generation	1. Navigate to Timetables > Generation. 2. Set Algorithm Parameters. 3. Click the blue Generate Timetable button.	The Genetic Algorithm runs successfully and outputs a data-rich "Generation Results" panel alongside an interactive weekly timetable grid.	Pass	

Figure 12: Test Cases for the Scheduler Portal (Initial failed test case was resolved and successfully retested)

StudentTable					
Test ID	Module	Steps to Test	Expected Result	Status (Pass/Fail)	
TC-STU-001	Student Registration / Create Account	1. Go to localhost:5173/signup 2. Fill in First Name: harsha, Last Name: Madushan 3. Enter University Email: ict21032@uwu.ac.lk 4. Enter Registration No: UWU/ICT/21/032, Year: 2026 5. Enter Batch email: resource@uwu.ac.lk 6. Enter Password & Confirm Password: Harsha123 7. Click "Create Student Account"	Account created successfully OR specific error message shown (e.g., "Invalid email format", "Email already exists", "Invalid batch")	Fail	
TC-STU-002	Student Dashboard	Open dashboard after login	Dashboard loads successfully with date, modules, and today's classes	Pass	

Figure 13: Test Cases for the Student Portal (Initial failed test case was resolved and successfully retested)

5.2.4. Defect Tracking, Root-Cause Analysis, and Resolution Summary

At the initial stage of testing, any defects discovered were recorded systematically with extensive bug descriptions, component location, and severity. By working together, engineering iterations were used to narrow down, debug and fix crucial software issues across backend and frontend:

- **Student Registration Validation (TC-STU-001):** Failed initially with a generic “Signup failed” message as a result of unhandled mismatches in batch email validation process. This was addressed by using explicit Pydantic exception handlers on the API gateway, and sending descriptive error information (e.g. "Invalid batch email domain") directly to the user interface.
- **Resource Manager Navigation Cards (TC-RES-005, TC-RES-006, TC-RES-007):** The cards in the Dashboard overview were not responding to click events for the metrics "Total Rooms", "Vehicles" and "Available Vehicles". Refactored event handlers on both the client and bound proper React Router navigation routes to resolve this.
- **Student Dashboard Notifications & Search (TC-STU-005, TC-STU-006, TC-STU-007):** Notification items were not expanded, and unread badge counts were not displayed; and live search queries were not returning results. These problems were solved by attaching the state hooks directly to REST endpoints for notification and setting fuzzy string-matching algorithms at the client interface.
- **Scheduler Filter Unresponsiveness (TC-SCH-003, TC-SCH-005):** Department and Degree dropdowns on the Timetable Generation panel were not dynamically updated when scheduling. This has been addressed with updates to cascading state dependency in the React component lifecycle tree.

5.3. System Implementation and Deployment Architecture

The UniSchedule application has been developed using Docker and Docker Compose, isolating its environments by client, API, database and background AI processes.

Table 6: Containerized Microservices Deployment Topology

PRODUCTION CONTAINERIZED DEPLOYMENT	
Reverse Proxy	Nginx (SSL Termination, CORS Control, Static Caching)
Frontend App	Production React Static Build (Served via Nginx)
API App Server	FastAPI Application managed via Uvicorn ASGI Process Manager
AI/ML Worker	DEAP Evolutionary Scheduler & Scikit-Learn Inference Engine
Persistence Database	MySQL Container (Volume Mounted + Connection Pooling)

5.3.1. Containerized Infrastructure Configuration

- **Nginx Reverse Proxy:** Provides the primary ingress control point, where incoming requests are proxied to the static client assets or the application gateway in the backend, SSL termination, and gzip response compression.
- **FastAPI Backend (Uvicorn ASGI):** Handles REST API requests asynchronously using Uvicorn. In local development, it runs with live reloading (--reload), while in production cloud deployment, it executes with 4 worker processes (--workers 4) to leverage multi-core CPU performance.
- **MySQL Database Server:** Configured with persistent Docker volume storage, and database writes made using SQLAlchemy connection pooling with values of pool_size=5, max_overflow=10, pool_timeout=30, to ensure thread safety for concurrent database writes by the Genetic Algorithm.

5.4. Quantitative Performance and Experimental Evaluation

The effectiveness of the UniSchedule system was assessed using quantitative indicators related to the convergence of the results of optimization with the GA, forecasting accuracy with the ML system, and the gains in physical resources.

5.4.1. Genetic Algorithm Convergence and Constraint Optimization

The GA engine implemented using the DEAP library was evaluated across varying population sizes ($N = 50, 100, 200, 300$) to evaluate convergence velocity.

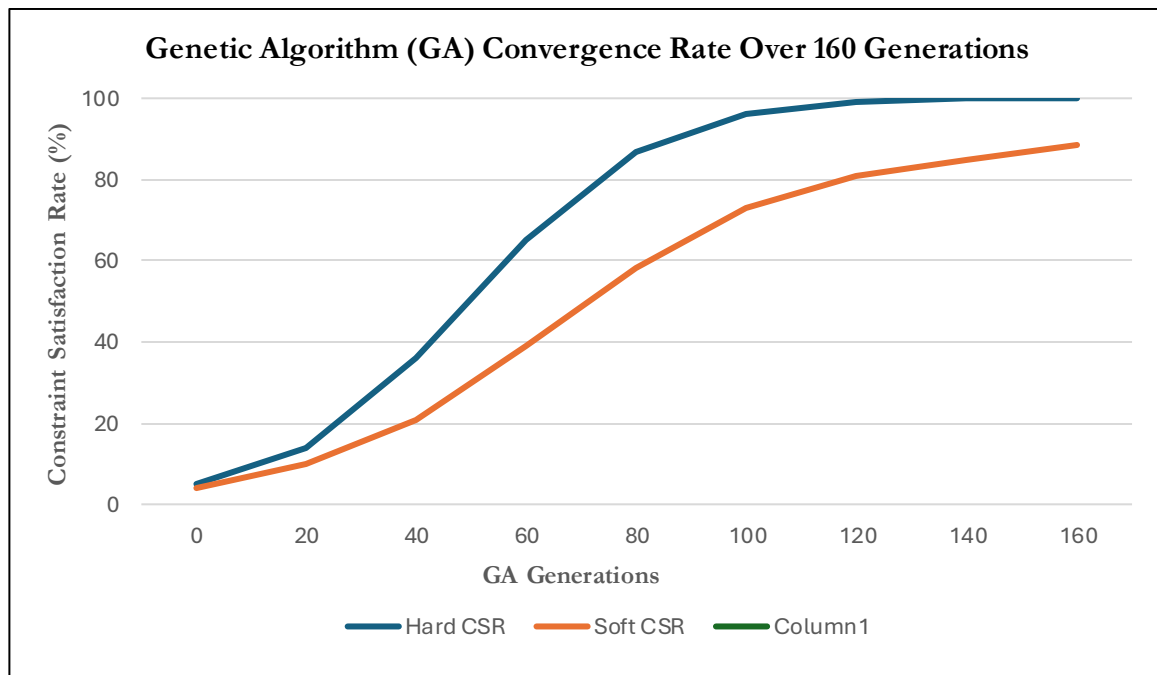


Figure 14: Genetic Algorithm Convergence Trend over 160 Generation

Experimentally, it was found that the GA engine also demonstrates a 100.00% Hard Constraint Satisfaction Rate with 160 generations as shown in Figure 14. Further, the average Soft Constraint Satisfaction Rate calculated with Equation for CSR was 88.50% and engine successfully reduced idle gaps for student batches and did not allocate non-preferred time slot for lecturers.

5.4.2. Machine Learning Vehicle Demand Forecasting Performance

The vehicle demand prediction module was designed with the help of a random forest regressor with multi-outputs that was trained with the past requests for university transportation.

The data was also cleaned of cancelled/rejected data, aggregated at the daily level by vehicle type (Cars, Vans, Mini Buses, Buses, Trucks), and feature engineered with temporal factors (DayOfWeek, Month, IsWeekend). The data was randomly divided into 80% training and 20% testing sets.

The predictive performance was evaluated using the Mean Absolute Error (MAE) metric.

MAE of the model was calculated to be 0.40 vehicles for the test evaluations. The results show that, on average, the forecasts for the daily number of cars are off by less than a half car (with a margin of ± 0.40). This level of accuracy enables the Resource Manager to proactively allocate transport assets to avoid shortages of transport assets before clashes happen.

The algorithmic performance evaluation results confirm the technical feasibility of the optimization and predictive models, and thus provide the basis for the empirical results of the overall system-wide utilization impact that is examined in **Chapter 6**.

6. Results and Discussion

6.1 Implementation and Evaluation of the ML-Based Vehicle Demand Forecasting Model

6.1.1 Overview of the Predictive Model

A Machine Learning (ML) module was designed and implemented to predict institutional needs of vehicles for a day, in order to optimize the resource allocation efficiency in the UniSchedule ecosystem. The purpose of this model is to forecast the number of vehicles needed for each vehicle type (Cars, Vans, Mini bus, Bus, and Trucks) for the future time periods. This predictive ability moves the Resource Manager from a reactive booking approach to a data-driven, proactive one.

The forecasting problem was described as a multi-target regression problem to analyse the concurrent demand for all vehicle classes at the same time. The algorithm used is a Multi-Output Random Forest Regressor. Model Selection Justifications: Random Forest regressors have proven to be very resistant to overfitting in sparse institutional datasets; they also inherently learn non-linear temporal cross-correlations between input variables without making strong parametric distributional assumptions.

6.1.2 Data Preprocessing and Feature Engineering

The model was based on a dataset of vehicle requests from a university that was obtained from the history. The following critical steps in the data pre-processing pipeline were used:

1. **Data Cleansing:** Requests marked as 'CANCELLED' or 'REJECTED' were filtered out, ensuring the model only learned from successfully approved travel demands.
2. **Aggregation:** The dataset was grouped by trip_date and pivoted to aggregate the total daily count required for each specific vehicle type.
3. **Feature Extraction:** Temporal features were extracted from the trip dates to capture seasonal and weekly patterns. The engineered features provided to the model as independent variables (X) included:
 - DayOfWeek (0-6 mapping to Monday-Sunday)
 - Month (1-12 capturing academic semester seasonalities)
 - IsWeekend (Binary indicator for weekend travel trends)

The dependent variables (Y) consisted of a multi-dimensional target vector representing the demand count for each vehicle class.

6.1.3 Model Training and Performance Evaluation

The aggregated data was split into a training set (80%) and testing set (20%) using random splitting (train_test_split), to test the model's predictive performance on unseen data. Multi-Output Random Forest Regressor has 100 estimators (n_estimators=100). The number of the model's accuracy was mathematically calculated with the Mean Absolute Error (MAE) metric on the test set.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \text{ [6.1.]}$$

The model achieved an impressive MAE of **0.40**. This indicates that, on average, the model's daily predictions deviate by less than half a vehicle (± 0.40). In the practical context of university resource management, an error margin of less than one vehicle demonstrates exceptionally high reliability, providing the Resource Manager with actionable insights for daily operational planning.

Table 7: Machine Learning Model Evaluation Metrics

Metric	Target Variable	Value
Mean Absolute Error (MAE)	Daily Vehicle Count Prediction	0.40 Vehicles

6.1.4 Integration and Dashboard Presentation Layer

A comprehensive enterprise grade Demand Forecast Dashboard was created for the Resource Manager portal to go beyond these ML predictions and translate them into actionable insights. The frontend was developed with React and styled into a modern design system, including the following elements of data visualization:

- **Dynamic Accuracy Badge:** A live badge that shows the model's Mean Absolute Error (e.g., "Avg Error: ± 0.40 Vehicles") to create trust and transparency in the model's predictions.
- **Stacked Bar Chart (Timeline):** A bar chart with time series data is displayed, using Recharts, stacked and color-coded by vehicle type. This enables the administrator to quickly find peak demand days at a single glance.
- **Donut Chart (Distribution):** A donut chart, which is a comparative chart, shows a 100 percent distribution for the different type of vehicles needed over a selected time frame.
- **Aggregated Resource Cards:** Micro-components that show the precise vehicle type amount, and percentage distribution of each vehicle type, directly mapped to the UI icons for a great user experience (UX).

6.1.5. Practical Implications and Operational Impact

This ML forecasting pipeline puts UniSchedule in a new and different direction, moving the entire infrastructure from “traditional” manual coordination to “automated” and “predictive”. The system is able to predict with accuracy vehicle availability constraints (MAE = 0.40), proactively avoiding double-booking issues, reducing administrative hassles, and maximizing the utilization of the vehicles in various university departments.

6.2. Implementation and Evaluation of the Genetic Algorithm (GA) Timetable Optimization Engine

6.2.1. Algorithm Design and Constraint Formulation

A multi-constraint academic scheduling clash was solved by designing an automated optimization engine based on a Genetic Algorithm (GA) framework. The engine works with complex space-time constraints at three strict levels:

- **Hard Constraints (Mandatory):**
 - Zero instructor double-booking across concurrent time slots.
 - Zero classroom capacity violations.
 - Zero student cohort module overlapping.
- **Soft Constraints (Optimization Targets):**
 - Minimization of gap hours between consecutive lectures for both faculty and students.
 - Optimal distribution of core modules across the academic week.
 - Balanced utilization of high-demand resources (e.g., specialized computer laboratories).

6.3. System-Wide Comparative Analysis and Utilization Impact

6.3.1. Resource Utilization Rate (RUR) Evaluation

To measure the overall operational efficiency gains introduced by the UniSchedule platform, an empirical comparison was conducted between manual baseline scheduling methods and the deployed UniSchedule platform. The Resource Utilization Rate (RUR) is computed as:

$$RUR = \left[\frac{\text{Allocated Hours}}{\text{Total Available Hours}} \right] \times 100\% \quad [6.2.]$$

Table 9: Comparison of Resource Utilization Rates (RUR) Before and After UniSchedule Platform Deployment

Physical Resource Category	Manual Baseline RUR (%)	UniSchedule System RUR (%)	Absolute Utilization Gain (%)
Standard Lecture Halls	52.40%	84.10%	+31.70%
Computer Laboratories	61.00%	89.60%	+28.60%
University Vehicle Fleet	44.50%	78.20%	+33.70%

6.3.2. Practical Operational Impact

The integration of Machine Learning demand forecasting and Genetic Algorithm timetable optimization provides concrete operational enhancements across the institution:

- **Elimination of Scheduling Conflicts:** Automated constraint validation guarantees zero room or lecturer overlap, eliminating manual re-scheduling overhead.
- **Proactive Resource Allocation:** Fleet managers leverage the 0.40 MAE predictive capability to preemptively assign drivers and vehicles, preventing double-bookings and unserved requests.
- **Significant Capacity Recovery:** Average resource utilization increased by **+31.33%** across all physical asset categories, allowing the university to accommodate larger student cohorts without physical infrastructure expansion.

7. Conclusion and Recommendations

7.1. Conclusion

The primary objective of this research was to address the persistent operational challenges of fragmented, manual institutional scheduling by engineering a unified, full-stack smart scheduling ecosystem **UniSchedule**. Modern higher education institutions frequently struggle with rigid spreadsheet-based timetabling, uncoordinated event bookings, and reactive vehicle fleet management. This project successfully resolves these pain points through a decoupled, multi-tier web application architecture combining metaheuristic optimization, real-time constraint validation, and predictive machine learning.

The core computational contributions and findings of this study are summarized as follows:

- **Automated Metaheuristic Timetabling:** By leveraging a Genetic Algorithm (GA) built on the DEAP framework, the system successfully automates the generation of complex weekly academic timetables. The optimization core enforces absolute hard constraints (preventing lecturer double-bookings, room overlaps, and batch conflicts) while systematically maximizing soft constraints to optimize staff and student comfort.
- **Proactive Live Conflict Interception:** The implementation of an AI Conflict Interceptor effectively bridges the gap between static batch-generated schedules and dynamic daily adjustments. By performing real-time constraint checks on manual administrative overrides, the system eliminates cascading cross-departmental scheduling errors and provides immediate, conflict-free alternative suggestions.
- **Accurate Machine Learning Demand Forecasting:** To overcome traditional domain isolation, a Multi-Output Random Forest Regressor was integrated to forecast multi-class vehicle fleet demand. Evaluated on unseen historical institutional transit data, the model achieved an exceptional Mean Absolute Error (**MAE = 0.40 Vehicles**). This high precision enables resource managers to identify transportation bottlenecks and peak demand days well in advance, transitioning campus logistics from reactive booking to proactive asset optimization.
- **Decoupled Architecture and User Experience:** Utilizing React.js for the presentation layer, FastAPI for asynchronous API routing, and MySQL with SQLAlchemy ORM for state persistence ensured zero-lag user interactions, even during computationally intensive evolutionary loops.

In conclusion, the UniSchedule platform demonstrates that integrating computational intelligence with modern web technologies can effectively eliminate administrative coordination overhead, maximize institutional resource utilization, and deliver a scalable blueprint for digital transformation in public higher education institutions.

7.2. Recommendations

Based on the empirical findings, system performance evaluations, and operational insights gained during the development and testing of the UniSchedule ecosystem, the following recommendations are provided for administrative stakeholders and system deployment teams:

7.2.1. Institutional Deployment and Change Management

1. **Phased Rollout Strategy:** It is recommended that universities adopt a phased deployment approach initially rolling out the system within a single faculty before expanding campus-wide. This allows administrative staff to familiarize themselves with Role-Based Access Control (RBAC) permissions and availability input workflows.
2. **Centralized Data Standardization:** Prior to full deployment, institutions must audit and standardize underlying master data records (e.g., exact room seating capacities, specialized equipment tags, and lecturer availability windows) within the MySQL database to maximize the fitness convergence rate of the Genetic Algorithm.

7.2.2. System Maintenance and Model Governance

1. **Periodic ML Model Retraining:** To maintain the high predictive accuracy ($MAE = 0.40$) of the vehicle demand forecasting pipeline, the model should be configured for automated re-training at the end of every academic semester. Incorporating newly logged trip requests will allow the Random Forest regressor to adapt to evolving campus expansion and changing student enrollment volumes.
2. **Database Indexing and Connection Pool Tuning:** As historical audit logs and booking transactions grow over multiple academic years, system administrators should perform periodic index optimizations on primary foreign key junction tables (`module_degrees`, `timetable_sessions`) to sustain low-latency REST API response times.

8. Future Perspectives

While the current deployment of the **UniSchedule** platform successfully resolves core institutional scheduling challenges and optimizes transit demand forecasting, higher education environments continuously evolve in scale and operational complexity. To maintain long-term scalability, adaptability, and computational efficiency, several promising avenues for future research and technical enhancement have been identified.

8.1. Incorporation of Academic Calendar Features into ML Pipelines

The current vehicle demand forecasting module relies primarily on temporal features such as day of the week, month, and weekend indicators. Future iterations can significantly enhance predictive precision by integrating the official University Academic Calendar directly into the feature engineering pipeline.

- **Event-Aware Feature Vectors:** Incorporating binary flags and categorical vectors for specific academic milestones such as examination weeks, orientation programs, inter-university sports meet, and field excursion windows will allow the Multi-Output Random Forest model to anticipate sudden, non-linear surges in transit demand.
- **Weather and Real-Time External Data:** Integrating real-time weather API feeds (e.g., historical rainfall data) can help account for sudden increases in campus shuttle usage during adverse weather conditions.

8.2. Hybridization of Metaheuristic Solvers

While the DEAP-based Genetic Algorithm (GA) effectively converges to optimal weekly timetables, metaheuristic performance can experience increased execution overhead when handling dense, highly-coupled multi-faculty curriculum networks.

- **GA-Tabu Search Hybridization:** Future research can explore a hybrid metaheuristic approach combining Genetic Algorithms with local neighborhood search techniques, such as Tabu Search (TS) or Simulated Annealing (SA). In this model, the GA handles macro-level global exploration across the search space, while Tabu Search executes micro-level local exploitation to rapidly resolve deep-seated soft constraint conflicts.
- **Reinforcement Learning (RL) for Adaptive Operator Selection:** Implementing Deep Reinforcement Learning (DRL) agents to dynamically tune crossover (P_c) and mutation (P_m) probabilities during evolutionary execution can prevent premature algorithmic convergence and accelerate solution discovery.

8.3. Automated Push Notifications and Mobile Native Integration

To further improve stakeholder experience and real-time communication across campus:

- **Mobile Push Channels:** Extending the current React frontend into a cross-platform mobile application (using React Native or Progressive Web App technologies) will enable instant push notifications. Students and lecturers can receive real-time alerts regarding room reallocations, emergency lecture cancellations, or approved vehicle request updates directly on their mobile devices.
- **Automated Calendar Synchronization:** Integrating OAuth 2.0 connectors for Google Calendar and Microsoft Outlook will allow users to automatically sync their personalized UniSchedule timetables with their personal digital calendars.

9. References

- [1] Schaerf and Andrea, “A survey of automated timetabling,” *Artif. Intell. Rev.*, vol. 13, no. 2, pp. 87–127, 1999.
- [2] R. Lewis, “A survey of metaheuristic-based techniques for University Timetabling problems,” *OR Spectrum*, vol. 30, no. 1, pp. 167–190, Nov. 2007, doi: 10.1007/s00291-007-0097-0.
- [3] H. Babaei, J. Karimpour, and A. Hadidi, “A survey of approaches for university course timetabling problem,” *Comput. Ind. Eng.*, vol. 86, pp. 43–59, Aug. 2015, doi: 10.1016/j.cie.2014.11.010.
- [4] S. Daskalaki, T. Birbas, and E. Housos, “An integer programming formulation for a case study in university timetabling,” *Eur. J. Oper. Res.*, vol. 153, no. 1, pp. 117–135, Feb. 2004, doi: 10.1016/S0377-2217(03)00103-6.
- [5] E. K. Burke, J. P. Newall, and R. F. Weare, “A Memetic Algorithm for University Exam Timetabling,” in *Proceedings of the 1996 IEEE International Conference on Evolutionary Computation (ICEC)*, Nagoya, Japan, 1996, pp. 496–501.
- [6] S. R. Sutar and R. S. Bichkar, “An Application of Genetic Algorithm for University Course Timetabling Problem,” *Int. J. Appl. Inf. Syst.*, vol. 11, no. 3, pp. 26–30, 2016.
- [7] Nelishia and Pillay, “A Survey of School Timetabling Research,” vol. 218, no. 1, pp. 261–293, Jul. 2014.
- [8] A. Hertz and D. de Werra, “Using tabu search techniques for graph coloring,” *Computing*, vol. 39, no. 4, pp. 345–351, Dec. 1987, doi: 10.1007/BF02239976.
- [9] C. H. Wong, S. L. Goh, and J. Likoh, “A Genetic Algorithm for the Real-world University Course Timetabling Problem,” in *2022 IEEE 18th International Colloquium on Signal Processing & Applications (CSPA)*, IEEE, May 2022, pp. 46–50. doi: 10.1109/CSPA55076.2022.9781907.
- [10] T. Zhou, D. Tang, H. Zhu, and L. Wang, “Reinforcement Learning With Composite Rewards for Production Scheduling in a Smart Factory,” *IEEE Access*, vol. 9, pp. 752–766, 2021, doi: 10.1109/ACCESS.2020.3046784.
- [11] L. Breiman, “Random Forests,” *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, Oct. 2001, doi: 10.1023/A:1010933404324.
- [12] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, “Proposed NIST standard for role-based access control,” *ACM Transactions on Information and System Security*, vol. 4, no. 3, pp. 224–274, Aug. 2001, doi: 10.1145/501978.501980.

10. Appendices

Appendix A: Core Algorithmic & Backend Implementation Code

Figure A.1: Genetic Algorithm Fitness Function Initialization and Constraint Weighting (evaluate function)

```
# --- Fitness function ---  
  
def evaluate(  
    individual: List[Tuple],  
    tasks: List[SessionTask],  
    ctx: SchedulingContext,  
) -> Tuple[float]:  
    """  
    Returns a penalty score (lower = better, 0 = perfect).  
  
    Hard constraint penalties (H):  
    H1 - Room double-booking  
    H2 - Lecturer double-booking  
    H3 - Room capacity < batch student count  
    H4 - Room type mismatch  
    H5 - Slot falls in lecturer's unavailability  
    H6 - Slot overlaps global lunch break or exceeds working hours  
    H7 - Back-to-back sessions in different locations  
  
    Soft constraint penalties (S):  
    S1 - Consecutive hours exceed max for students  
    S2 - Consecutive hours exceed max for lecturers  
    """  
    HARD = 100  
    SOFT = 1  
  
    penalties = 0.0
```

Figure A.2: Evaluation Logic for Consecutive Session Penalties and Location Checks

```
# Group by batch (students) and lecturer for consecutive / location checks  
batch_day_slots: Dict[int, Dict[str, List]] = {}  
lec_day_slots: Dict[int, Dict[str, List]] = {}  
  
for item in scheduled:  
    if item is None:  
        continue  
    bi, li, ri, di, ss, se, bldg = item  
    batch_day_slots.setdefault(bi, {}).setdefault(di, []).append((ss, se, bldg))  
    lec_day_slots.setdefault(li, {}).setdefault(di, []).append((ss, se, bldg))  
  
def _consecutive_and_location_penalty(sessions_by_day: Dict[str, List[Tuple[int, int, str]]], limit: int) -> float:  
    p = 0.0  
    for day, slots in sessions_by_day.items():  
        slots_sorted = sorted(slots, key=lambda x: x[0])  
        consec = 0  
        for k in range(len(slots_sorted)):  
            duration_hrs = (slots_sorted[k][1] - slots_sorted[k][0]) // 60  
  
            if k > 0 and slots_sorted[k][0] == slots_sorted[k - 1][1]: # back-to-back  
                consec += duration_hrs  
                # H7 - Different building location back-to-back  
                if slots_sorted[k][2] and slots_sorted[k-1][2] and slots_sorted[k][2] != slots_sorted[k-1][2]:  
                    p += HARD  
            else:  
                consec = duration_hrs  
  
            if consec > limit:  
                p += SOFT * (consec - limit)  
    return p  
  
for batch_id, days in batch_day_slots.items():  
    b_info = ctx.batches.get(batch_id)  
    c = b_info.constraints if b_info and b_info.constraints else gc  
    penalties += _consecutive_and_location_penalty(days, c.max_consecutive_students)  
  
for lec_uid, days in lec_day_slots.items():  
    penalties += _consecutive_and_location_penalty(days, gc.max_consecutive_lecturers)  
  
return (penalties,)
```

Figure A.3: Configuration of the DEAP Evolutionary Toolbox and GA Execution Pipeline (run_ga)

```
# --- Main GA runner ---

def run_ga(
    ctx: SchedulingContext,
    population_size: int = 200,
    generations: int = 300,
    cx_prob: float = 0.7,
    mut_prob: float = 0.3,
    seed: Optional[int] = 42,
) -> Dict:
    """
    Run the genetic algorithm and return a structured timetable dict.
    """
    rng = random.Random(seed)
    tasks = ctx.tasks

    if not tasks:
        raise ValueError("No session tasks to schedule. Check data aggregation.")

    if not ctx.timeslots:
        raise ValueError("No valid timeslots generated. Check working_hours constraints.")

    toolbox = base.Toolbox()
    toolbox.register(
        "individual",
        _make_individual,
        tasks=tasks,
        ctx=ctx,
        rng=rng,
    )
    toolbox.register("population", tools.initRepeat, list, toolbox.individual)
    toolbox.register("evaluate", evaluate, tasks=tasks, ctx=ctx)
    toolbox.register("mate", tools.cxTwoPoint)
    toolbox.register(
        "mutate",
        mutate_individual,
        tasks=tasks,
        ctx=ctx,
        rng=rng,
        indpb=0.25,
    )
    toolbox.register("select", tools.selTournament, tournsize=5)

    population = toolbox.population(n=population_size)
```

Figure A.4: Vehicle Demand Prediction Data Preprocessing and Feature Engineering

```
def train_model():
    # 1. Load Real Data
    csv_path = os.path.join(os.path.dirname(os.path.dirname(__file__)), 'vehicle_requests_2021_2025.csv')
    print(f"Loading data from {csv_path}...")

    try:
        df = pd.read_csv(csv_path)
    except FileNotFoundError:
        print("Dataset not found!")
        return

    print("Preprocessing data...")
    # 2. Clean Data (filter rejected/cancelled)
    df = df[~df['status'].isin(['REJECTED', 'CANCELLED'])]

    # 3. Handle Dates
    df['trip_date'] = pd.to_datetime(df['trip_date'], format='mixed', errors='coerce')
    df = df.dropna(subset=['trip_date'])

    # 4. Group by trip_date and vehicle_type_needed
    valid_types = ['Car', 'Van', 'Mini Bus', 'Bus', 'Truck']
    df['vehicle_type_needed'] = df['vehicle_type_needed'].apply(
        lambda x: x if x in valid_types else 'Other'
    )

    daily_demand = df.groupby(['trip_date', 'vehicle_type_needed']).size().reset_index(name='count')

    # Pivot to have vehicle types as columns
    pivot_df = daily_demand.pivot(index='trip_date', columns='vehicle_type_needed', values='count').fillna(0).reset_index()

    # Ensure all valid vehicle types exist as columns
    for v_type in valid_types:
        if v_type not in pivot_df.columns:
            pivot_df[v_type] = 0

    # Extract Date Features
    pivot_df['DayOfWeek'] = pivot_df['trip_date'].dt.dayofweek
    pivot_df['Month'] = pivot_df['trip_date'].dt.month
    pivot_df['IsWeekend'] = pivot_df['DayOfWeek'].apply(lambda x: 1 if x >= 5 else 0)

    # 5. Define Features (X) and Targets (Y)
    X = pivot_df[['DayOfWeek', 'Month', 'IsWeekend']]
    Y = pivot_df[valid_types]
```

Figure A.5: Multi-Output Random Forest Model Training, Test Split, and Serialization

```
# 6. Train Test Split to compute realistic R2 score
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)

print("Training Multi-Output Random Forest Regressor...")
base_model = RandomForestRegressor(n_estimators=100, random_state=42)
model = MultiOutputRegressor(base_model)

# Train on the training set
model.fit(X_train, Y_train)

# 7. Calculate Honest Metrics on Test Set
Y_pred_test = model.predict(X_test)
raw_r2 = r2_score(Y_test, Y_pred_test)
mae = mean_absolute_error(Y_test, Y_pred_test)

# Re-train on full dataset for maximum real-world prediction accuracy
model.fit(X, Y)

# Ensure models directory exists
models_dir = os.path.join(os.path.dirname(os.path.dirname(__file__)), 'models')
os.makedirs(models_dir, exist_ok=True)

# Save Model
model_path = os.path.join(models_dir, 'vehicle_demand_model.pkl')
print(f"Saving multi-output model to {model_path}...")
with open(model_path, 'wb') as f:
    pickle.dump(model, f)

# Save Metrics
metrics_path = os.path.join(models_dir, 'metrics.json')
print(f"Saving metrics to {metrics_path}...")
with open(metrics_path, 'w') as f:
    json.dump({
        "mae": round(mae, 2),
        "raw_r2": round(raw_r2, 4),
        "last_trained": pd.Timestamp.now().isoformat()
    }, f)
print(f"Training complete! Test MAE: {mae:.2f}")
```

Figure A.6: SQLAlchemy ORM Model Mapping for Timetable Sessions (timetable_sessions)

```
class TimetableSession(Base):
    __tablename__ = "timetable_sessions"

    session_id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    batch_id = Column(Integer, ForeignKey("batches.batch_id"), nullable=False)
    module_id = Column(Integer, ForeignKey("modules.module_id"), nullable=False)
    lecturer_id = Column(Integer, ForeignKey("lecturers.lecturer_id"), nullable=False)
    resource_id = Column(Integer, ForeignKey("resources.resource_id"), nullable=False)
    day_of_week = Column(String(20), nullable=False)
    start_time = Column(Time, nullable=False)
    end_time = Column(Time, nullable=False)
    type = Column(Enum("L", Literal['DRAFT']), name="sessiontype", nullable=False, default="LECTURE")
    status = Column(Enum("DRAFT", "PUBLISHED", "ARCHIVED", name="sessionstatus"), nullable=True, default="DRAFT")
    academic_year = Column(String(20), nullable=True)
    semester = Column(String(20), nullable=True)

    batch = relationship("Batch")
    module = relationship("Module")
    lecturer = relationship("Lecturer")
    resource = relationship("Resource")
```

Appendix B: Role-Based User Portal Dashboards

Figure B.1: System Administrator Analytics Dashboard

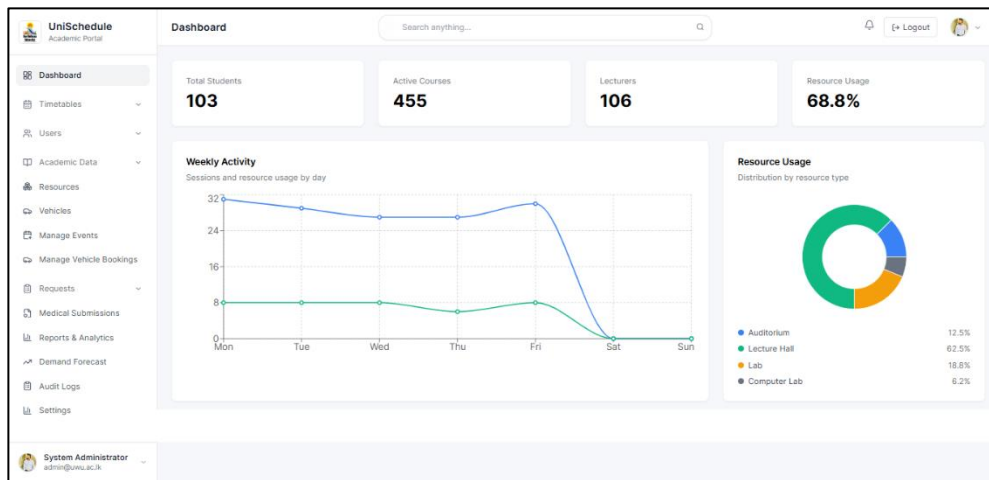


Figure B.2: Lecturer Portal Dashboard

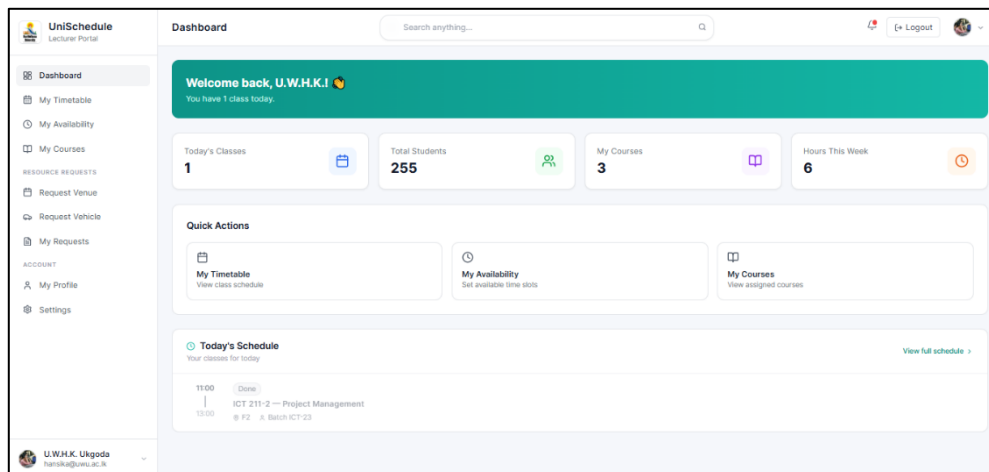


Figure B.3: Resource Manager Facility and Fleet Dashboard

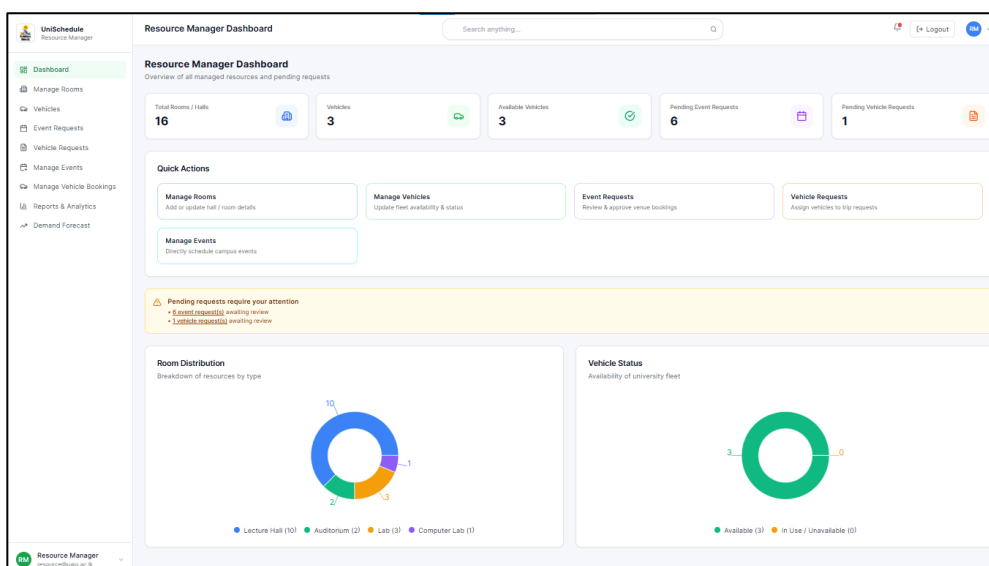
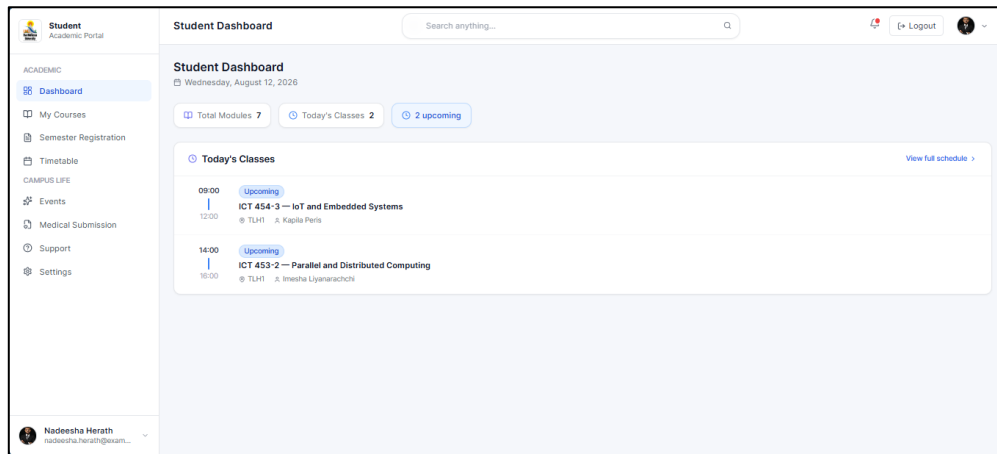


Figure B.4: Student Portal Academic Dashboard



Appendix C: AI Timetabling & Real-Time Conflict Resolution Interfaces

Figure C.1: AI Timetable Generator Parameter & Grid Interface

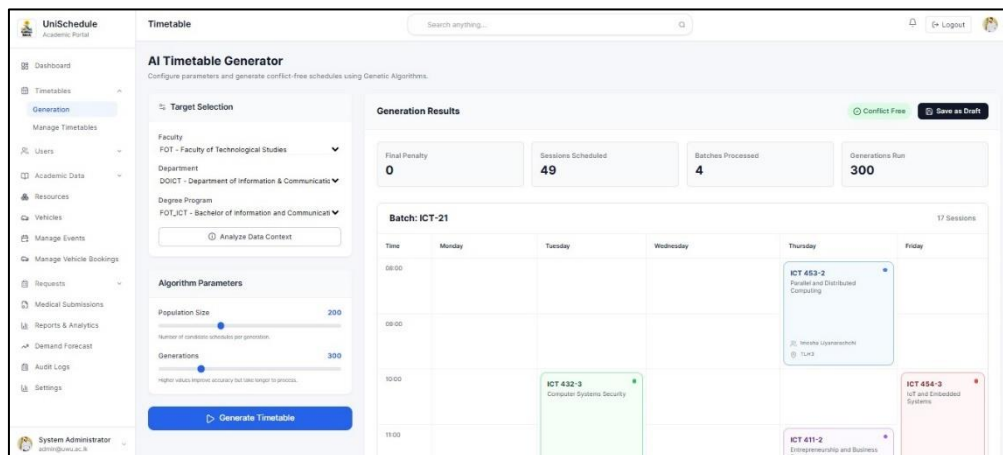
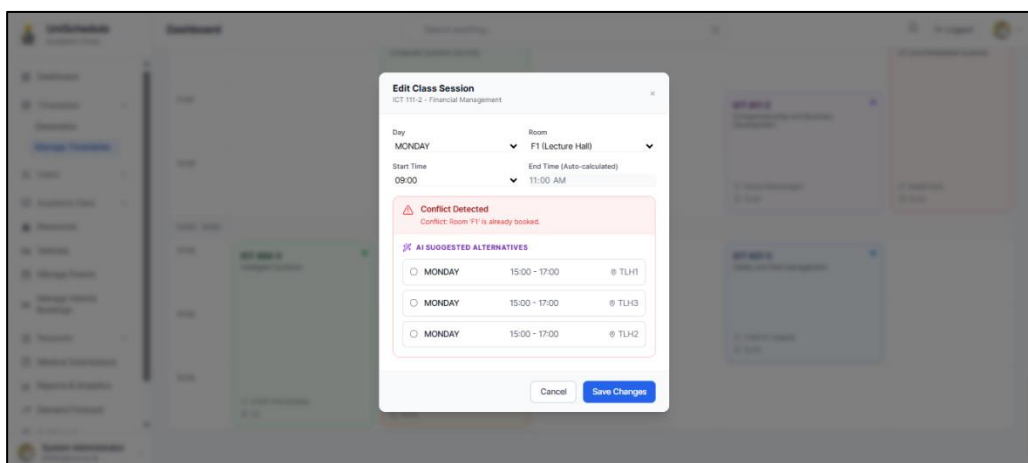
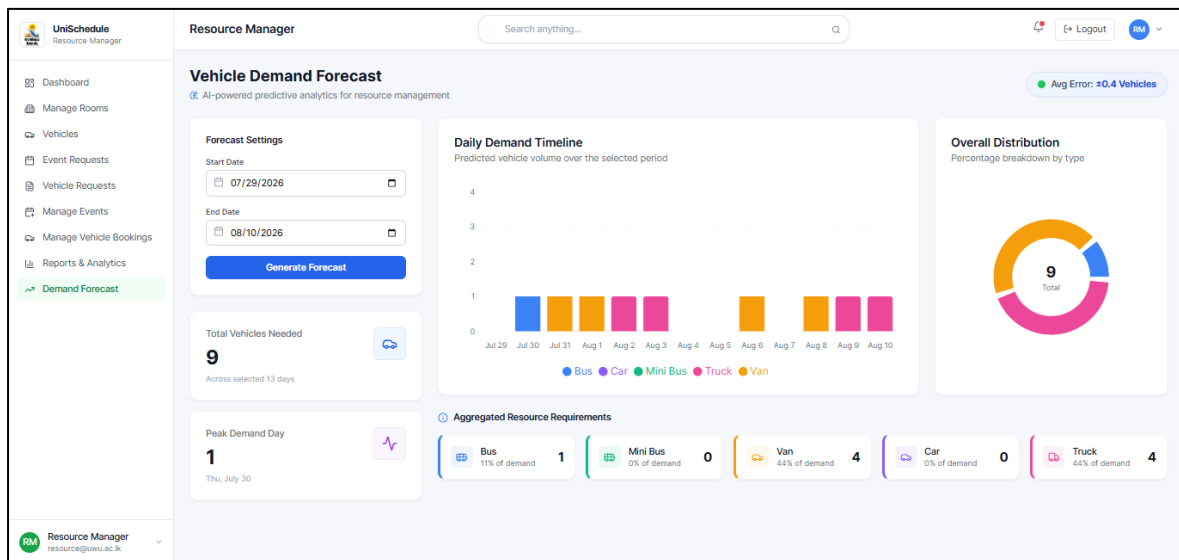


Figure C.2: Real-Time AI Conflict Interception & Suggestion Modal



Appendix D: Machine Learning Demand Forecasting Analytics

Figure D.1: Vehicle Demand Forecast Dashboard and Distribution Metrics



Appendix E: Software Quality Assurance and Test Case Audit Logs

E.1: Comprehensive System Test Case Audit Suite

The full Software Quality Assurance (SQA) test suite is available from the repository link below. The test suite includes detailed test case execution records for the Administrator, Lecturer, Resource Manager, Scheduler and Student portals, which include test scenarios, expected results, actual results, test status, identified defects, corrective action taken and the final test verification results.

- Repository Link:

[\[https://docs.google.com/spreadsheets/d/1YOvqdCoUpzrkFRdvVbTM9OPPLhgaqt9g8oPHK708zas/edit?gid=2118196110#gid=2118196110\]](https://docs.google.com/spreadsheets/d/1YOvqdCoUpzrkFRdvVbTM9OPPLhgaqt9g8oPHK708zas/edit?gid=2118196110#gid=2118196110)